Chapter 3

TECHNICAL BACKGROUND

Project Description

BrewManager is an offline-first mobile Point-of-Sale and reactive inventory management system developed specifically for local independent coffee shops. The system runs entirely on a single Android tablet device using a locally embedded SQLite database, which means it does not require an internet connection to perform its core functions. The application is built using Flutter configured for Android tablet landscape layout, and all data is stored and processed on the device itself through a centralized database helper class.

The development of BrewManager follows a ground-up, dependency-based build sequence where no module is constructed before the components it relies on are already in place. This approach ensures that each phase of development is stable before the next one builds on top of it.

The process begins with the project foundation, where the Flutter project is configured, the SQLite database schema is defined, and the state management and navigation routing structures are set up. A default Manager PIN is also seeded at this stage so the application is not locked on first use. The system requires this PIN to be changed on first launch. From there, the PIN-based authentication system is built to separate Manager and Barista

access before any feature screens are developed, since every other part of the system sits behind this role gate.

Once authentication is working, the back-office CRUD modules are constructed so that the Manager can configure ingredients, menu items, and recipe blueprints. These records are the foundation that every other feature depends on. The inventory receiving module is built alongside them, allowing the Manager to register incoming stock as inventory batches. Each batch holds one expiration date, one container capacity, and the number of containers received with that date, so a single receiving session may produce several batch records where dates or container sizes differ.

After the data layer is populated, the First-Expired First-Out engine is built and tested in isolation as a pure backend logic module, covering single-batch deductions, multi-batch splits, and zero-stock edge cases with unit tests.

With the FEFO engine verified, the POS transaction screen is developed on top of it, allowing baristas to take orders, trigger automatic ingredient deductions, generate on-screen digital receipts, record mid-shift losses through the Declare Container Lost function, and reverse mistaken orders through the Manager-authorized void function. Following the POS, the Proactive Expiry Monitoring Matrix is added to evaluate batch shelf-life against the device clock and feed normalized expiry risk scores to the later phases.

The End-of-Shift Reconciliation module is then built. This module computes the theoretical quantity of every active batch, accepts a physical count from staff through a sealed container tally and a fractional estimate of what

remains open, computes the variance between the two, and writes the corrected value back to the batch record. Blind audit conditions are enforced so that theoretical values are hidden until the count is submitted.

The final functional phases add the rule-based decision support module for waste prevention, the descriptive analytics dashboard with fl_chart visualizations, the manual CSV and PDF export feature, and the database backup and restore function. The project concludes with a structured system evaluation phase using ISO 25010 quality attributes, the System Usability Scale questionnaire, controlled accuracy testing against known quantities, and a comparison of end-of-shift procedure duration before and after deployment. The raw materials and tools used throughout development are listed below:

Component	Specification
Development Framework	Flutter SDK (Android tablet, landscape layout)
Programming Language	Dart
Local Database	SQLite via database_helper.dart
State Management	Provider or Riverpod
Chart Library	fl_chart
Target Device	Android tablet
IDE	Android Studio or Visual Studio Code
Version Control	Git
Export Formats	CSV, PDF
Backup Format	SQLite database file copy
Evaluation Instruments	ISO 25010, System Usability Scale

	(SUS), controlled accuracy testing, timed procedure comparison

Table 3.1 Raw Materials and Tools Table

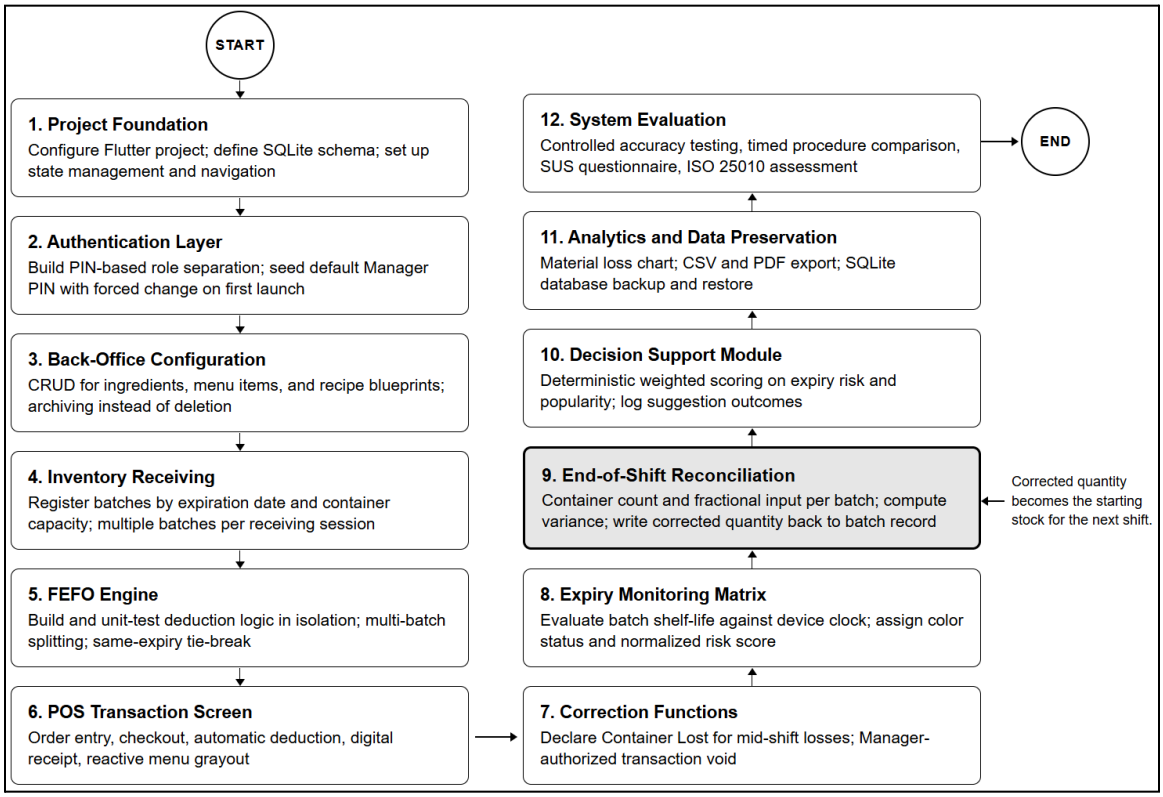


Figure 3.1 Project Development Flowchart

System Development Life Cycle (SDLC)

The development of BrewManager follows the Incremental Model as its System Development Life Cycle. This model was selected because the system is constructed in separate, functional stages where each increment builds directly on top of what was already completed and verified. This approach suits the architecture of BrewManager well, since the core inventory logic cannot function without the database and ingredient records being in place first, the POS screen cannot operate without a working FEFO engine beneath it, the reconciliation module cannot compute a variance without recorded transactions to produce a

theoretical quantity, and the analytics dashboard has no meaningful data to present without completed reconciliation logs. Because every phase depends on the one before it, delivering the system in ordered increments is both practical and structurally sound.

The development of BrewManager is organized into five increments. Each increment produces a functional component of the complete system, and each is aligned to a specific objective of this study.

Increment 1: System Foundation and Security Layer

The first increment establishes everything the rest of the system is built on. This covers the creation of the Flutter project configured for Android tablet landscape layout, the setup of the SQLite database helper class, and the definition of the complete database schema covering all tables including users, ingredients, menu items, recipe blueprints, inventory batches, shift sessions, transactions, transaction items, batch deductions, reconciliation logs, loss events, and suggestion logs. State management and navigation routing structures are also configured at this stage.

A default Manager PIN is seeded into the database so the application is not locked on first launch. The system requires this PIN to be changed before any other function becomes available, and PINs are stored as hashes rather than plain text.

With the foundation in place, the PIN-based authentication system is then developed. This includes building the numeric keypad entry screen, querying the users table upon PIN submission, detecting the role associated with the entered

PIN, and routing Manager accounts to the back-office dashboard while directing Barista accounts to the POS screen. An idle session lock is also implemented so that the application automatically returns to the PIN entry screen after a period of inactivity, with role-based routing applied again upon re-entry.

This increment produces no user-facing features on its own. It exists to guarantee that every later module operates on a stable schema behind an enforced role gate.

Increment 2: Back-Office Configuration, Inventory Receiving, and FEFO Engine

The second increment builds the data and logic layer that all transactional features depend on. It delivers the foundation of the first specific objective.

The Manager back-office screens are developed here, covering full CRUD operations for ingredient profiles, menu items, and recipe blueprints. Records that are referenced by completed transactions are archived rather than permanently deleted, so that historical sales and deduction data remain accurate. A uniqueness constraint is applied to recipe blueprint entries so that the same ingredient cannot be registered twice against a single menu item.

The inventory receiving module is developed alongside the configuration screens. The Manager registers incoming stock by recording each distinct combination of expiration date and container capacity present in a purchase, together with the number of containers matching that combination. Each combination produces a separate inventory batch record holding the expiry date, the capacity of a single container, the number of containers received, and the

total quantity remaining. No consolidation or approximation of expiration dates is applied. Because container capacity is stored at the batch level rather than on the ingredient, the same ingredient may be held in different container sizes at the same time.

After the configuration and receiving modules are in place, the First-Expired First-Out engine is developed and tested as an isolated backend module before being connected to any screen. This engine queries all active batches associated with a given ingredient, orders them by expiration date from earliest to latest, and deducts the required recipe quantity starting from the earliest-expiring batch. Where two batches share an expiration date, the batch holding the smaller remaining quantity is depleted first, so that partially used batches are closed out before full ones are opened.

When a batch does not have sufficient quantity to cover the full requirement, the engine fully depletes it and carries the remaining amount over to the next available batch. This multi-batch splitting logic continues until the full recipe quantity is satisfied. Every deduction is recorded in the batch deductions table with a reference to the originating transaction, and the quantity remaining in each affected inventory batch is updated accordingly. Recording deductions at this level of detail is what later allows a voided transaction to return each quantity to the exact batch it came from.

Unit tests are written to cover single-batch deductions, multi-batch splits, the same-expiry tie-break, and the zero-stock edge case before this engine is considered complete.

Increment 3: POS Transactions and Expiry Monitoring

The third increment connects the backend logic built in Increment 2 to the front-of-house interface. It completes the first specific objective and delivers the second.

The POS transaction screen is built first, presenting the menu grid to the Barista, graying out menu items whose dependent ingredients have reached zero stock, and updating this status reactively in real time the moment any batch is fully depleted. The Barista can build an order, view a running total, and proceed to checkout, at which point the FEFO engine is triggered for every item in the order. The entire checkout operation is wrapped in a single database transaction, so that a failure at any point rolls back every deduction rather than leaving ingredients deducted for a sale that was never recorded.

A digital on-screen receipt is generated, and the transaction is saved to the transactions and transaction items tables. Each transaction item stores the selling price at the moment of sale, so that later changes to a menu item's price do not alter the recorded value of past transactions.

Two correction functions are built into this screen. The Declare Container Lost function allows staff to record a mid-shift physical loss, such as a dropped container, by selecting the affected batch and recording how many containers were lost in quarter-container steps. The system deducts that amount from the batch and writes a loss event record, which is kept separate from the unexplained variance measured during reconciliation. The Transaction Void function, which requires Manager authorization, reverses a completed sale by

reading its batch deduction records and returning each quantity to the specific batch it was originally taken from.

Following the POS, the Proactive Expiry Monitoring Matrix is developed. This module queries all active inventory batches, compares each batch's expiration date against the current device date, and assigns a color-coded status: green for batches with more than seven days remaining, yellow for batches expiring within seven days, and red for batches expiring within 48 hours or already expired. Each batch is also assigned a normalized expiry risk score, where red receives 1.0, yellow receives 0.5, and green receives 0.0, which is later used by the decision support module.

A device clock check is implemented at this stage. The system stores the latest timestamp it has recorded and displays a warning if the device clock is subsequently found to be earlier than that value, since a backward clock change would cause expiry evaluation and FEFO routing to operate on incorrect dates.

Increment 4: End-of-Shift Reconciliation and Decision Support

The fourth increment delivers the third and fourth specific objectives. It contains the correction loop that distinguishes this system from one that only deducts.

The End-of-Shift Reconciliation module is developed first. A shift session is opened when a Barista signs in and closed when reconciliation is submitted, so that a single calendar day may contain more than one reconciliation.

The reconciliation process is initiated by the Barista or Manager through a dedicated shift-close button. At shift close, the system computes the theoretical

quantity of every active batch from the inventory batches table and presents an audit interface listing one block per batch. Each block displays the ingredient name, the batch expiration date, and the batch container capacity, so that staff can identify which physical containers belong to which batch.

Each batch accepts two inputs. The first is a count of sealed, unopened containers. The second is the combined amount remaining across all opened containers of that batch, entered through a stepper control in quarter-container increments. This control is not bounded at a single container, so staff submit one combined value regardless of how many containers are open, and the number of inputs per batch stays constant. Each control is initialized at the value recorded during the previous shift, so that routine closings require only small adjustments rather than entry from zero.

To prevent bias, theoretical quantities are hidden from staff until the audit is submitted and locked. One validation is applied during input: where a submitted sealed count exceeds the total containers recorded for that batch, the system requests confirmation before accepting the value, since such a value is physically impossible rather than merely unexpected. If confirmed, the system updates the batch container count, records the surplus as probable unrecorded receiving, and flags it for Manager review.

After submission, the system multiplies the submitted values by the batch container capacity to obtain the audited quantity, computes the variance against the theoretical quantity, and writes the corrected value directly to the batch record. This write-back is the operation that closes the correction loop, since the

quantity the FEFO engine reads at the start of the next shift is the quantity that was physically counted rather than an estimate carried forward. Every variance is saved to the reconciliation logs table with a reference to the batch and the shift session, and is categorized so that unexplained variance, declared loss, and unrecorded receiving remain distinguishable in later analysis.

A post-submission check compares variances across batches of the same ingredient. Where one batch returns a surplus and another returns a deficit of comparable magnitude, the system flags the pattern as a probable deviation from expiry-order stock rotation, since this signature indicates that material was taken from a batch other than the one the FEFO engine deducted from.

The rule-based decision support module is built after reconciliation, since it operates on corrected batch quantities. The module calculates a popularity score for each menu item by counting how frequently it appears in transaction items over the trailing seven days and normalizing the result against the highest-selling item. It then retrieves the highest expiry risk score among all ingredients in each menu item's recipe blueprint. These two values are combined using a weighted formula in which expiry risk accounts for 60 percent and popularity accounts for 40 percent. Where no sales history exists, as occurs during the first seven days of deployment, the module falls back to ranking by expiry risk alone.

The resulting scores rank available menu items, and the top-ranked items are displayed on the POS screen together with the expiry condition that triggered them, updating automatically after each completed transaction. Every

suggestion issued is written to a suggestion log along with whether the suggested item was subsequently sold, so that the module's effect on spoilage can be measured during evaluation.

Increment 5: Analytics, Data Preservation, and System Evaluation

The fifth and final increment completes the system with the descriptive analytics dashboard, the data preservation features, and the formal evaluation phase. It delivers the fifth specific objective.

The analytics dashboard is developed for the Manager and displays material loss over time, drawn from reconciliation logs and filterable by date range and ingredient. Loss is presented by category so that unexplained variance can be distinguished from declared losses such as dropped containers. Surpluses arising from unrecorded receiving are excluded from loss totals, since these represent record corrections rather than material waste.

Two data preservation features are then built. The manual export feature allows the Manager to generate CSV and PDF reports covering sales, inventory, and reconciliation history, saved to the tablet's internal storage for record-keeping outside the application. Separately, a database backup function copies the complete SQLite database file to external or user-selected storage, with a corresponding restore function that loads a saved database file back onto a device. Unlike the report exports, this copy preserves the full relational state of the system and can be used to rebuild operations following device loss or failure.

Once all functional phases are complete, the system evaluation is conducted. Test scripts are prepared covering the core workflows including POS

transactions, FEFO deductions with multi-batch splitting, transaction voiding, reconciliation audits with variance write-back, expiry alerts, and decision support suggestions.

The evaluation combines four instruments. Controlled accuracy testing establishes whether the system's computed quantities are correct, by loading known starting quantities, executing a scripted sequence of transactions, and comparing the system's output against independently calculated expected values. Timed procedure comparison measures the duration of the establishment's existing end-of-shift inventory routine before deployment and the duration of the BrewManager reconciliation process after deployment. The System Usability Scale questionnaire is administered to participating coffee shop staff and owners. The system is additionally evaluated against ISO 25010 quality attributes covering functional suitability, reliability, performance efficiency, and usability. Collected results across all four instruments are documented for inclusion in the final capstone report.

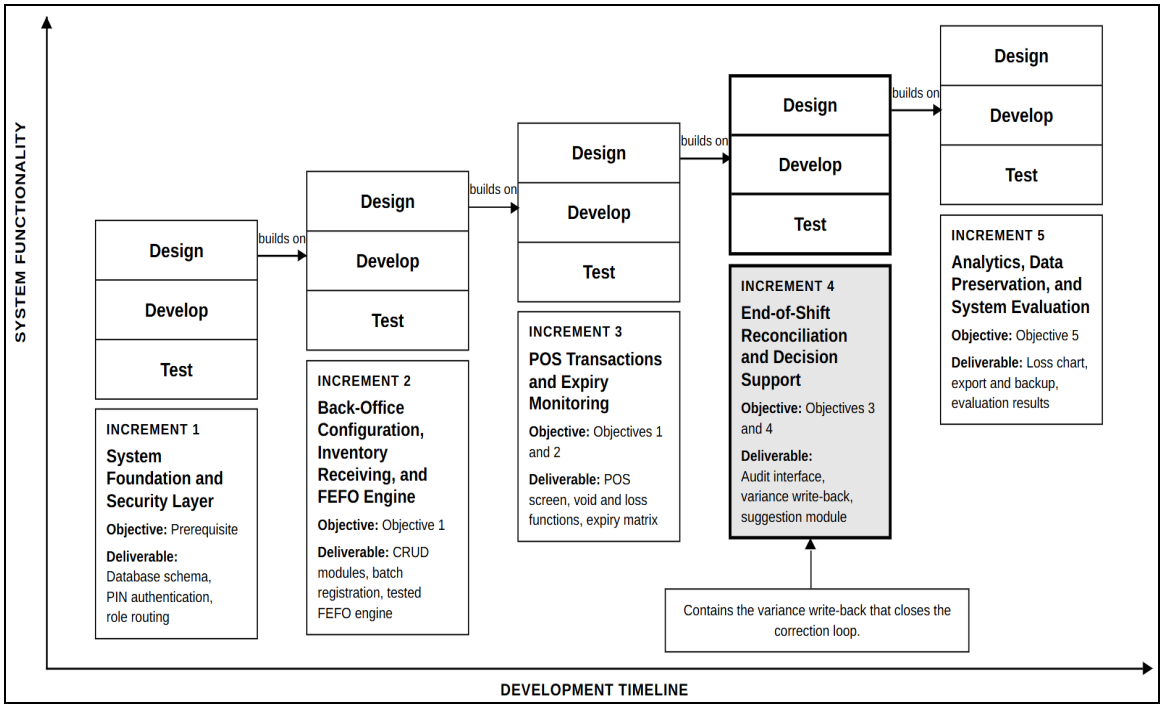


Figure 3.2 Incremental Model

Environment

Locale

The client for this study is Small House Cafe, an independent local coffee shop situated at 15 Tandang Sora Avenue, Sangandaan, Quezon City, 1116 Metro Manila. The cafe operates as a single-branch establishment and serves as the primary deployment environment for the BrewManager system. Because the system is designed with an offline-first, single-device architecture, no additional branches or remote access points are involved. All transactions, inventory records, and reconciliation data are managed on one Android tablet device located within the cafe premises.

Population of the Study

The population of this study consists of the individuals who interact with the BrewManager system during its deployment, testing, and evaluation. Because the system is designed for single-branch, single-device operation, the study follows a single-case deployment at Small House Cafe, and the population reflects the actual staffing of that establishment.

The administrator population consists of one owner who also functions as the Manager of the cafe. This user has full access to the system's back-office features, including ingredient and menu item configuration, recipe blueprint management, inventory receiving, the expiry monitoring dashboard, the analytics dashboard, transaction voiding, and the data export and backup functions. The Manager account is also the only account authorized to add or modify Barista PINs.

The front-of-house population consists of three staff members who work as cashiers and baristas and who share a single role within the system. These users operate the POS transaction screen to process customer orders, record mid-shift losses where they occur, and perform the end-of-shift reconciliation audit at the close of each shift. Because the cafe operates on rotating shifts, more than one reconciliation may be recorded within a single calendar day, and the system supports this through shift session records. The establishment therefore has four system users in total, all of whom participate in the study.

In addition to these four users, a minimum of three information technology practitioners or faculty members with system development experience participate

as evaluators of the system's functional suitability, reliability, and performance efficiency, since these attributes require technical observation rather than end-user experience.

The customers of Small House Cafe are not direct users of the BrewManager system, as the application does not include a customer-facing interface. Because the system operates entirely behind the counter, no customer data is collected, and customers are not counted within the study population. They are indirect beneficiaries, since FEFO routing, expiry monitoring, and reactive stock management help ensure that orders are fulfilled using safe and available ingredients.

Organizational Chart

The organizational structure of Small House Cafe reflects a lean, owner-operated setup typical of independent local coffee shops. At the top of the structure is the Owner, who also serves as the Manager responsible for all back-office decisions, inventory oversight, and system configuration. Reporting directly to the Owner are three staff members working as Cashiers and Baristas, who handle all front-of-house operations including order-taking, drink preparation, mid-shift loss recording, and end-of-shift inventory audits. These staff work on rotating shifts rather than simultaneously. This two-level structure corresponds directly to the two access levels enforced by the system's permission matrix, where the Owner holds Manager access and all front-of-house staff share Barista access. Figure 3.3 below illustrates this structure.

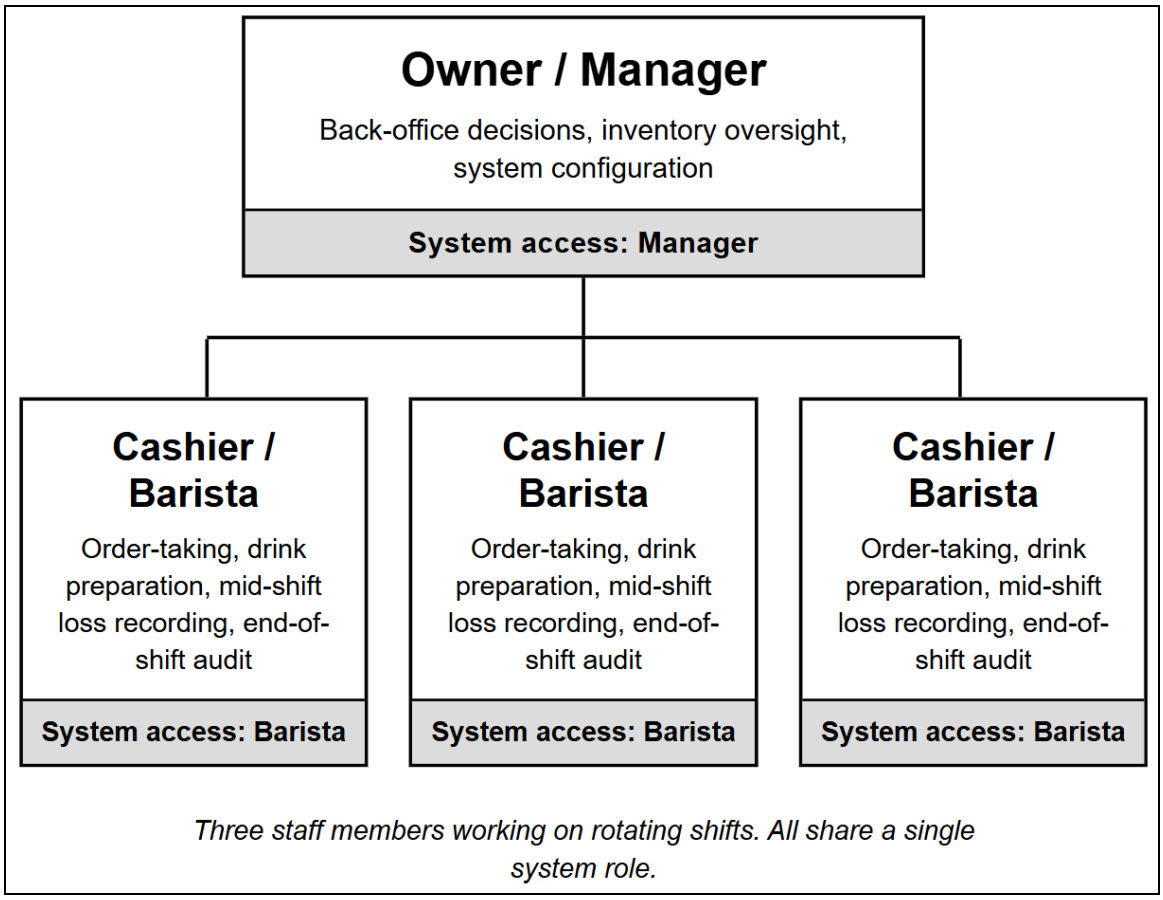


Figure 3.3 Organizational Chart

Requirements Modeling

The requirements of the BrewManager system are described through five categories: input, process, output, performance, and control. These categories define what data the system receives, what it does with that data, what it produces, how well it must perform, and what rules govern access and behavior.

Input

The inputs to the system are divided between the two user roles defined by the permission matrix.

Under the Manager role, inputs include the configuration data needed to set up and maintain the system. These are ingredient profiles containing the ingredient name and unit of measurement; menu item records containing the item name and selling price; and recipe blueprints that map a specific quantity of each ingredient to a menu item. The Manager also enters inventory receiving data. For each purchase, the Manager records every distinct combination of expiration date and container capacity present in that purchase, together with the number of containers matching each combination. Each combination produces one inventory batch record. A single receiving session may therefore create several batch records where expiry dates or container sizes differ. The Manager additionally supplies date range and ingredient parameters when filtering the analytics dashboard, and provides authorization when a transaction void is requested.

Under the Barista role, inputs consist of the customer orders entered through the POS screen during a shift, mid-shift loss declarations, and the physical count submitted at shift close. Audit input is collected per batch and consists of two values. The first is a count of sealed, unopened containers belonging to that batch. The second is the combined amount remaining across all opened containers of that batch, entered through a stepper control in quarter-container increments. This control is not limited to a single container, so one combined value is submitted regardless of how many containers are open. No weighing, unit conversion, or arithmetic is required from the Barista at any point.

Process

Once inputs are received, the system executes several automated processes entirely within the device's local SQLite database without requiring an internet connection.

When a Barista completes a checkout, the Automated Recipe Mapping Engine is triggered. This engine reads the recipe blueprint for each item in the order, identifies all active batches associated with the required ingredients, and sorts them by expiration date from earliest to latest. Where two batches share an expiration date, the batch holding the smaller remaining quantity is placed first. The engine then deducts the required quantity beginning with the earliest-expiring batch. If that batch does not hold enough to cover the full requirement, the engine fully depletes it and carries the remaining amount to the next batch in order. This multi-batch splitting continues until the full recipe quantity is satisfied. Every deduction is recorded in the batch deductions table with a reference to the originating transaction, and the entire checkout is executed as a single database transaction so that a failure at any point rolls back all deductions.

At the same time, the system evaluates each active inventory batch against the current device date to determine shelf-life status. Batches with more than seven days remaining are marked green, batches expiring within seven days are marked yellow, and batches expiring within 48 hours or already past their expiration date are marked red. Each status is converted to a normalized expiry risk score for use by the decision support module.

During end-of-shift reconciliation, the system multiplies the Barista's submitted container count and fractional value by the registered container capacity of that batch to obtain the audited quantity. It computes the variance by comparing this figure against the theoretical quantity the system recorded through automated deductions, then writes the audited quantity back to the batch record as the corrected value. Two additional checks are performed. A submitted count exceeding the total containers recorded for a batch is flagged as probable unrecorded receiving. Two batches of the same ingredient returning variances that offset each other are flagged as a probable deviation from expiry-order stock rotation.

When a transaction void is authorized, the system reads the batch deduction records associated with that transaction and returns each deducted quantity to the specific batch it was originally taken from, then removes the transaction from sales records.

The rule-based decision support module counts how often each menu item has appeared in completed transactions over the trailing seven days and normalizes this into a popularity score. It combines this with the highest expiry risk score among the item's recipe ingredients using a weighted formula of 60 percent expiry risk and 40 percent popularity, then ranks all currently available menu items by the resulting score. Where no sales history exists, ranking falls back to expiry risk alone.

Output

The outputs of the system are the results and information presented to each user after the processes above are completed.

On the Barista's POS screen, the system outputs a real-time menu grid that grays out items whose dependent ingredients have reached zero stock, an on-screen digital receipt generated after each completed checkout, and a ranked list of waste-prevention suggestions displayed together with the expiry condition that triggered each one, updating automatically after every transaction. At the close of a shift, the system presents the computed variance for each batch to the Barista after the physical count has been submitted and locked, along with any unrecorded receiving or stock rotation flags raised during the process.

On the Manager's dashboard, the system outputs a color-coded expiry monitoring matrix showing the shelf-life status of all active inventory batches, current stock balances, and a material loss chart showing variance per ingredient over time from reconciliation logs, filterable by date range and ingredient. Loss is presented by category so that unexplained variance can be distinguished from declared losses such as dropped containers.

For data preservation, the system produces CSV and PDF reports covering sales, inventory, and reconciliation history for manual export to the tablet's internal storage, and a complete copy of the SQLite database file for backup to external storage.

Performance

The system must perform reliably within the operational constraints of a single Android tablet device running an embedded SQLite database without internet connectivity.

All core processes, including checkout, FEFO deduction, and menu grayout, must execute quickly enough that the Barista does not experience noticeable delay during a transaction. The reactive menu grayout must update the moment any batch reaches zero stock, not only at the point of checkout. Decision support scoring and suggestion display must refresh after each completed transaction so that suggestions are always based on current data.

The end-of-shift reconciliation interface must present all active batches without perceptible loading delay, since the audit is performed at closing when staff are working to finish. The audit is designed so that a complete reconciliation can be submitted in a small number of interactions per batch, and reducing the duration of this procedure relative to the establishment's existing manual routine is a measured outcome of this study.

The system must also remain stable during continuous use across a full operating shift without data loss or application failure. Because all data resides on a single device with no automatic synchronization, the database backup function must produce a file that can be restored onto a replacement device with no loss of relational integrity.

Control

The system enforces control through a PIN-based User Access Control permission matrix that strictly separates what each role can see and do. Baristas are restricted to the POS transaction screen, the mid-shift loss declaration, and the end-of-shift reconciliation audit interface. They cannot access ingredient configurations, recipe blueprints, inventory receiving, the analytics dashboard, transaction voiding, or the data export and backup functions. These features are reserved exclusively for the Manager account. PINs are stored as hashes rather than plain text, and the default Manager PIN seeded at installation must be changed before any other function becomes available.

Blind audit enforcement is applied as a control measure during reconciliation. The system does not display the theoretical quantity to the Barista before the physical count is submitted. The theoretical value is revealed only after submission is locked, preventing the count from being adjusted to match the system's expectation. One narrow exception applies. Where a submitted sealed container count exceeds the total containers recorded for that batch, the system requests confirmation before accepting the value. This validation identifies a physically impossible entry rather than an unexpected one, and does not disclose the theoretical quantity.

Transaction voiding is controlled through Manager authorization. Because a void reverses both a sale and its associated inventory deductions, allowing it without approval would permit front-of-house staff to alter sales and inventory

records without oversight. Voids are recorded rather than deleted, so that the reversal itself remains visible in the transaction history.

FEFO routing acts as an automated control over ingredient usage, ensuring that no transaction consumes a later-expiring batch while an earlier-expiring batch still holds stock. Because the system routes deductions in software but cannot control which physical container staff open, deviation from expiry-order withdrawal is treated as a condition to be detected during reconciliation rather than as an assumption. Where the Declare Container Lost function is used, the system deducts the specified number of containers from the identified batch and records the event as a declared loss, kept separate from the unexplained variance measured at shift close.

Finally, because the system operates entirely offline, the device's internal clock serves as the sole time reference for expiry calculations and FEFO routing. The system retains the latest timestamp it has recorded and displays a warning if the device clock is subsequently found to be earlier than that value. It cannot independently verify the correct time without network access, which represents a control boundary that must be maintained by the cafe's management.

Requirements Specification

Operational Feasibility

Operational feasibility examines whether the proposed system can realistically function within the day-to-day environment of the client. For Small House Cafe, the assessment confirms that BrewManager is operationally viable

because it is designed around the actual workflow of a two-role, single-device coffee shop.

The system does not require internet connectivity, so service continues during network outages that would halt a cloud-based point-of-sale system. It does not depend on external hardware beyond the Android tablet itself. The reconciliation module in particular was designed to avoid any dependence on weighing equipment, since requiring a digital scale would impose a purchase on the client and introduce a device that could fail or be misplaced. Instead, the audit records stock as a count of sealed containers and a fractional estimate of what remains open, which requires no equipment and no calculation from staff.

The system also fits the labor available at the establishment. The most time-sensitive procedure, the end-of-shift audit, is performed at closing when staff are working to finish. It is therefore limited to two inputs per active batch, with each control initialized at the previous shift's value so that routine closings require only small adjustments. Reducing the duration of this procedure relative to the cafe's existing manual routine is a measured outcome of this study.

The PIN-based role separation ensures that each user sees only the screens relevant to their tasks, which keeps the interface focused for both the Manager and the Barista. Because all front-of-house staff share a single Barista role, no additional configuration is required when shift assignments change.

Adoption risk is addressed by keeping the front-of-house workflow close to the existing one. Baristas continue to take orders and count stock at closing as they already do. What changes is that the counting is entered into a guided

interface rather than written on paper, and that the deduction of ingredients happens automatically rather than being recorded manually or not at all.

System Flowchart

The system flowchart presents the overall operational flow of BrewManager from application launch through role authentication to the completion of a shift. It shows the two role paths diverging after PIN entry, the transaction and deduction sequence performed during service, and the reconciliation sequence performed at shift close, including the write-back of corrected quantities to inventory records. Figure 3.4 presents this flow.

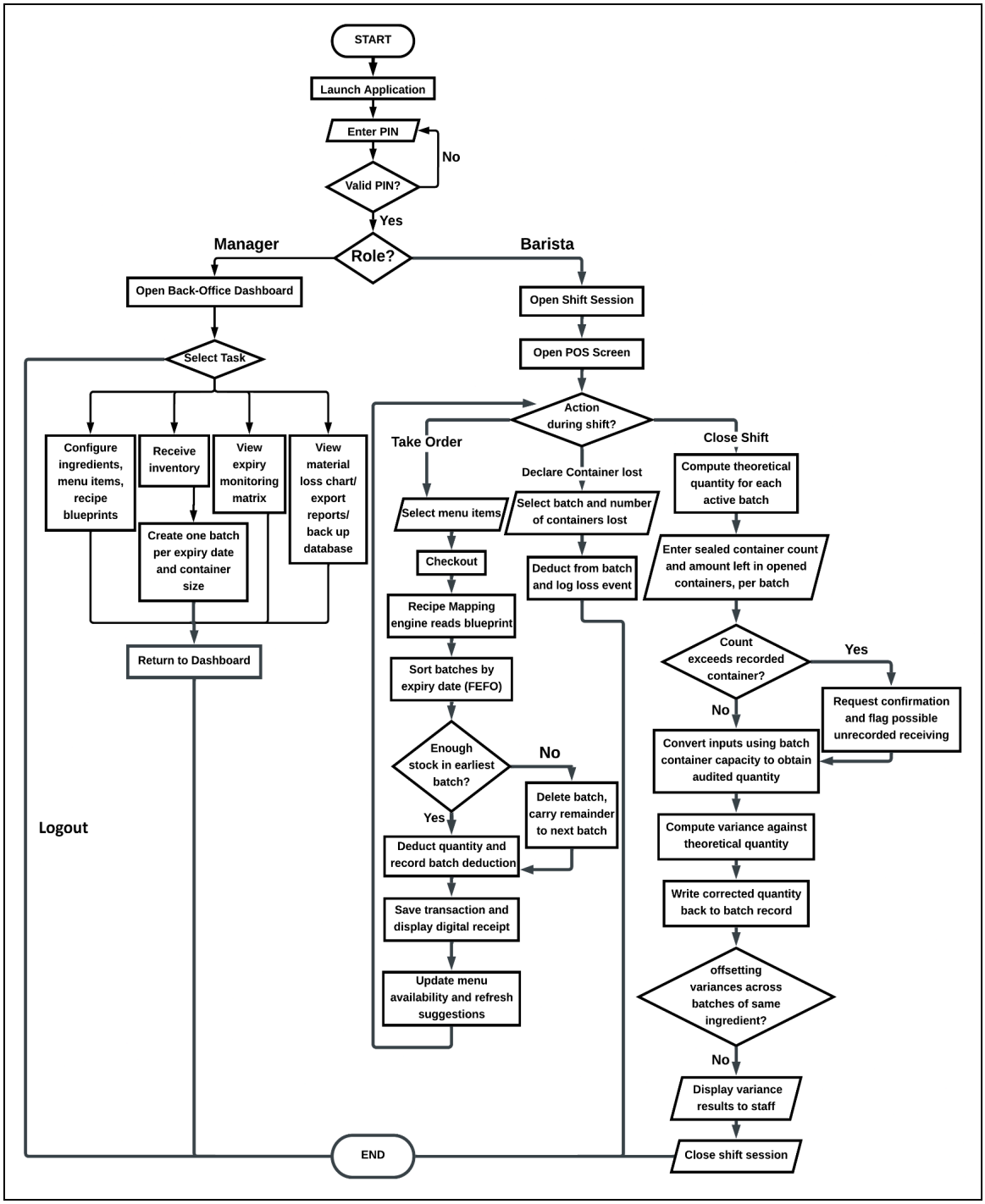


Figure 3.4 System Flowchart

Site Map

The site map presents the screen hierarchy of the application and shows which screens are reachable from each role after authentication. The Manager

path leads to the back-office dashboard and its configuration, receiving, monitoring, analytics, and data preservation screens. The Barista path leads to the POS transaction screen and the end-of-shift reconciliation interface. The separation shown in the site map corresponds directly to the permission matrix described under Control. Figure 3.5 presents this hierarchy.

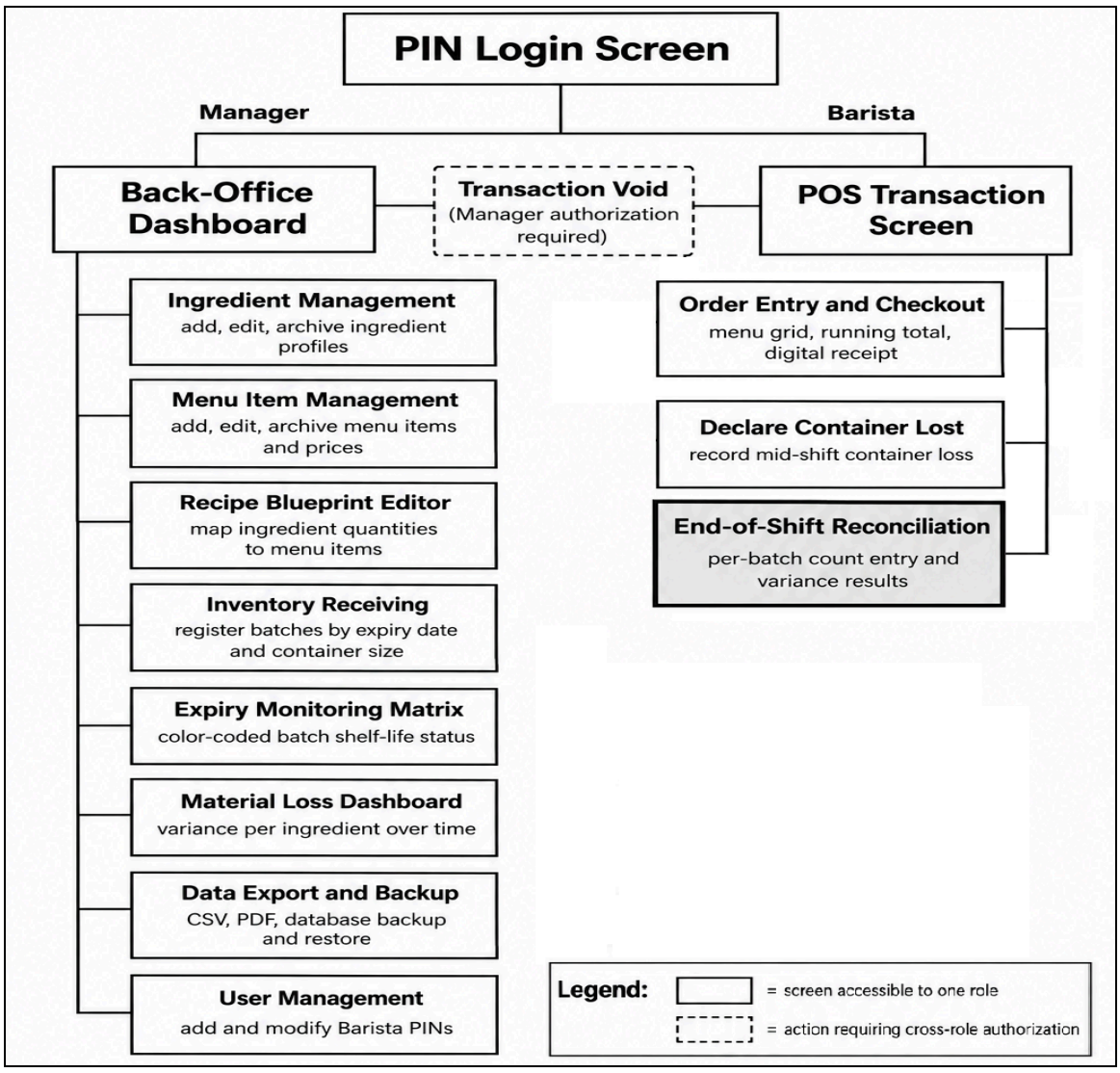


Figure 3.5 Site Map

User Journey Map

The user journey map traces the sequence of actions each role performs across a full operating day. The Manager journey covers opening configuration and inventory receiving, periodic monitoring of expiry status, and end-of-day review of reconciliation results and material loss. The Barista journey covers shift sign-in, order processing during service, recording of any mid-shift losses, and completion of the closing audit. Figure 3.6 presents both journeys.

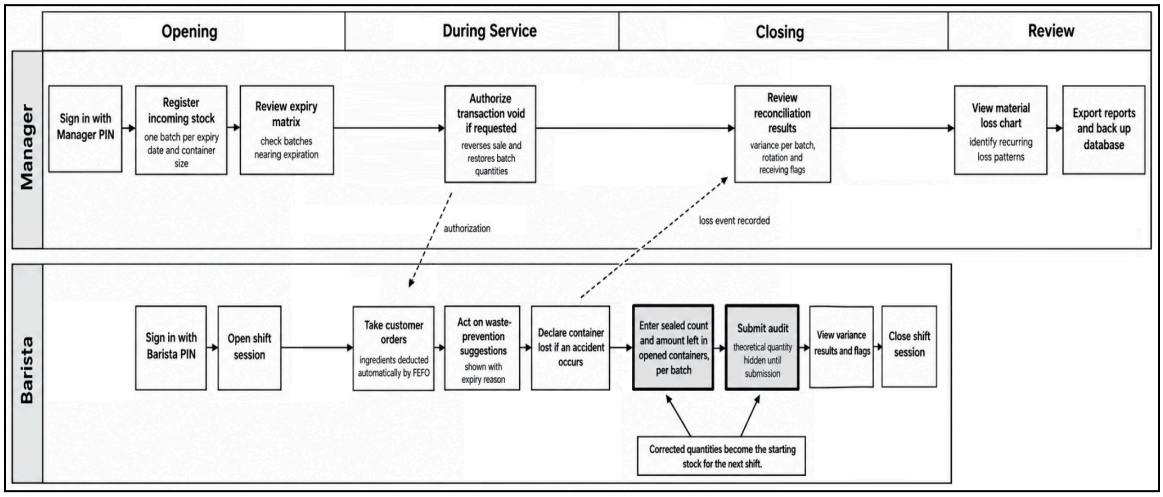


Figure 3.6 User Journey Map

Schedule Feasibility

Schedule feasibility assesses whether the BrewManager system can be completed within the time available for the capstone development cycle. The assessment covers the full period from initial project proposal through system implementation and evaluation at the client site.

The development sequence follows the five increments described earlier in this chapter. Because each increment depends on the one before it, the schedule is arranged so that no increment begins before its prerequisites are

complete and tested. The foundation and security layer must be finished before any feature screens are built. The FEFO engine must be tested on its own before the POS screen is connected to it. Transaction records must exist before reconciliation has a theoretical quantity to audit against. This chain of dependencies sets the order of the schedule and limits how much work can be done at the same time.

Two scheduling concerns needed particular attention. The first is recipe blueprint configuration, which depends on getting accurate ingredient measurements from the client. Independent coffee shops often prepare drinks by practiced estimation rather than by measured amounts, so written recipe quantities could not be assumed to exist. Time was therefore set aside for on-site observation and measurement of actual pours across a representative set of drinks. This activity runs alongside development rather than after it, since the deduction engine cannot be tested properly without accurate blueprints.

The second concern is the evaluation phase, which requires the system to run at the client site long enough to build up reconciliation records. Material loss patterns cannot be seen from a single shift, and the analytics dashboard needs several reconciliation cycles before it shows anything meaningful. The schedule therefore allows deployment time before the evaluation activities begin, rather than treating implementation and evaluation as one stage.

Table 3.2 presents the planned and actual timelines for all project activities from proposal through evaluation. Figure 3.7 presents the same information as a

Gantt chart, with planned durations shown against actual durations so that any deviations can be identified.

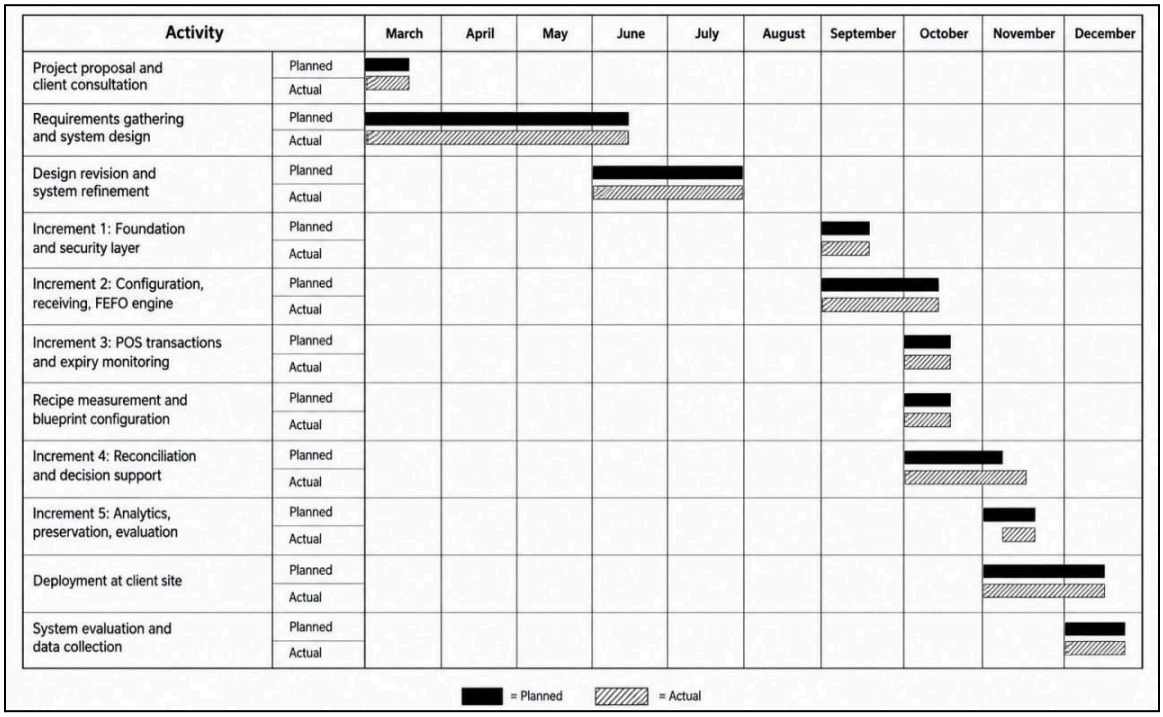


Figure 3.7 Gantt Chart

Economic Feasibility

Economic feasibility examines whether the cost of building and deploying BrewManager is reasonable compared to the value it gives back to the client. The assessment covers development costs, hardware costs, ongoing costs, and the savings the system is expected to produce.

Cost and Benefit Analysis

The development cost of BrewManager is low because the system is built entirely on free and open-source tools. Flutter, SQLite, and the supporting Dart packages have no licensing fees. The proponents carry out the development as

part of the capstone requirement, so there is no cost for hired labor. Version control and project hosting use free-tier services.

The main cost to the client is hardware. The system needs one Android tablet that can run the application in landscape orientation. Because the system works offline with a local database and does not perform heavy computation, a mid-range device is enough. A suitable tablet is available for roughly PHP 8,000 to 12,000. No server, network equipment, or subscription service is needed.

The system was designed to avoid extra hardware costs wherever possible. The reconciliation module records stock through container counts and fractional estimates rather than through weight, so the client does not need to buy a digital scale. Receipts are shown on screen rather than printed, so there is no cost for a receipt printer or for paper. These were deliberate design choices, made so that the total cost to the client stays limited to a single device.

Ongoing costs are minimal. The system does not use cloud hosting or synchronization, so there are no monthly fees. The electricity used to charge the tablet is very small next to the cafe's normal operating costs. Data is preserved through the built-in export and backup functions, which write to storage already on the device or to an inexpensive external drive.

The benefits fall into three areas. The first is less material waste. The expiry monitoring matrix, FEFO routing, and the waste-prevention suggestions work together to help ingredients get used before they spoil. Perishable ingredients such as milk and syrups make up a large share of what a coffee shop buys regularly, so even a small drop in spoilage adds up over time.

The second is less work at closing. The end-of-shift audit replaces manual counting and recording with two guided inputs per batch. Reducing the time this takes is a measured outcome of this study, and time saved at closing is time returned to other tasks or to staff going home earlier.

The third is better purchasing decisions. The material loss dashboard shows which ingredients are lost most often and in what amounts. This gives the owner information that manual tracking does not provide, which makes it possible to spot and correct over-ordering.

Cost Recovery Scheme

BrewManager is developed as an academic capstone project and given to the client at no charge, so there is no purchase price to recover. The cost recovery scheme therefore covers the client's hardware spending rather than a software fee.

The tablet is the only real expense the client takes on. This cost is recovered through the reduction in material waste the system is designed to produce. Because the system measures material loss directly through reconciliation logs, the client can see the value of ingredients lost over a given period and compare it against what the tablet costs. The system therefore supplies the data needed to check its own recovery rather than relying on an estimate.

No recurring costs are passed to the client, since the system needs no subscription, no hosting, and no licensed software. If the tablet ever needs

replacing, the database backup function allows the system to be restored onto a new device with the historical records intact, so the client does not start over.

Data and Process Modelling

This section describes how data is stored and how it moves through the BrewManager system. Because the application keeps all records in a local SQLite database on a single device, the data model must hold every relationship the system depends on without relying on an external server. Four models are presented: the entity relationship diagram, the data dictionary, the context diagram, and the data flow diagram.

Entity Relationship Diagram

The entity relationship diagram shows the tables in the BrewManager database and how they are linked. The design is built around three ideas that shape the rest of the system.

The first is that inventory is tracked at the batch level rather than at the ingredient level. An ingredient record holds only the name and unit of measurement. The actual stock lives in inventory batch records, each of which holds one expiration date, the capacity of a single container, the number of containers received, and the quantity remaining. One ingredient can therefore have many batches at the same time, each with its own expiry date and container size. This is what allows FEFO routing to choose between batches and what allows reconciliation to correct each batch separately.

The second is that every deduction is recorded individually. When a sale takes ingredients from one or more batches, each deduction is written to its own record linked to both the transaction and the batch it came from. Storing deductions this way is what makes a transaction void possible, since the system can read these records and return each amount to the exact batch it was taken from.

The third is that reconciliation results are stored per batch rather than per ingredient. Each reconciliation log records the theoretical quantity, the audited quantity, the variance between them, and the type of variance found. Because the log is tied to a batch and to a shift session, the system can show which batch drifted and during which shift, rather than only reporting a total for the ingredient.

The main entities and their relationships are as follows:

Users holds the accounts for the Manager and Barista roles, with PINs stored as hashes. One user can open many shift sessions.

Shift Sessions records the opening and closing time of each shift and the user who opened it. One shift session contains many transactions and many reconciliation logs. This allows more than one reconciliation to be recorded in a single calendar day.

Ingredients holds the profile of each raw material, including its name, unit of measurement, and archive status. One ingredient has many inventory batches and appears in many recipe blueprints.

Inventory Batches holds the stock itself. Each record belongs to one ingredient and stores an expiration date, container capacity, the number of containers

received, and the quantity remaining. One batch can be the source of many batch deductions and can have many reconciliation logs across different shifts.

Menu Items holds the drinks offered for sale, including the name, selling price, and archive status. One menu item has many recipe blueprint entries and appears in many transaction items.

Recipe Blueprints links a menu item to the ingredients it needs and the quantity of each. A uniqueness rule prevents the same ingredient from being added twice to one menu item.

Transactions holds each completed sale, with a reference to the shift session it belongs to, the time of sale, the total amount, and whether the sale has been voided. One transaction contains many transaction items.

Transaction Items holds each line of a sale, linking a transaction to a menu item and storing the quantity ordered and the selling price at the moment of sale. Storing the price here keeps past sales accurate even if the menu price is changed later.

Batch Deductions records each individual deduction, linking a transaction to the specific batch the quantity was taken from. One transaction can produce several deductions when multi-batch splitting occurs.

Reconciliation Logs holds the result of each audited batch, linking a shift session to an inventory batch and storing the theoretical quantity, audited quantity, variance, and variance type.

Loss Events records mid-shift losses declared through the Declare Container Lost function, linking a shift session to a batch and storing the amount lost.

These are kept separate from reconciliation logs so that known losses and unexplained variance can be told apart in the analytics.

Suggestion Logs records each waste-prevention suggestion the decision support module produces, along with the menu item suggested, the expiry condition that triggered it, and whether the item was sold afterwards. This is what allows the module to be evaluated.

Figure 3.8 presents the entity relationship diagram.

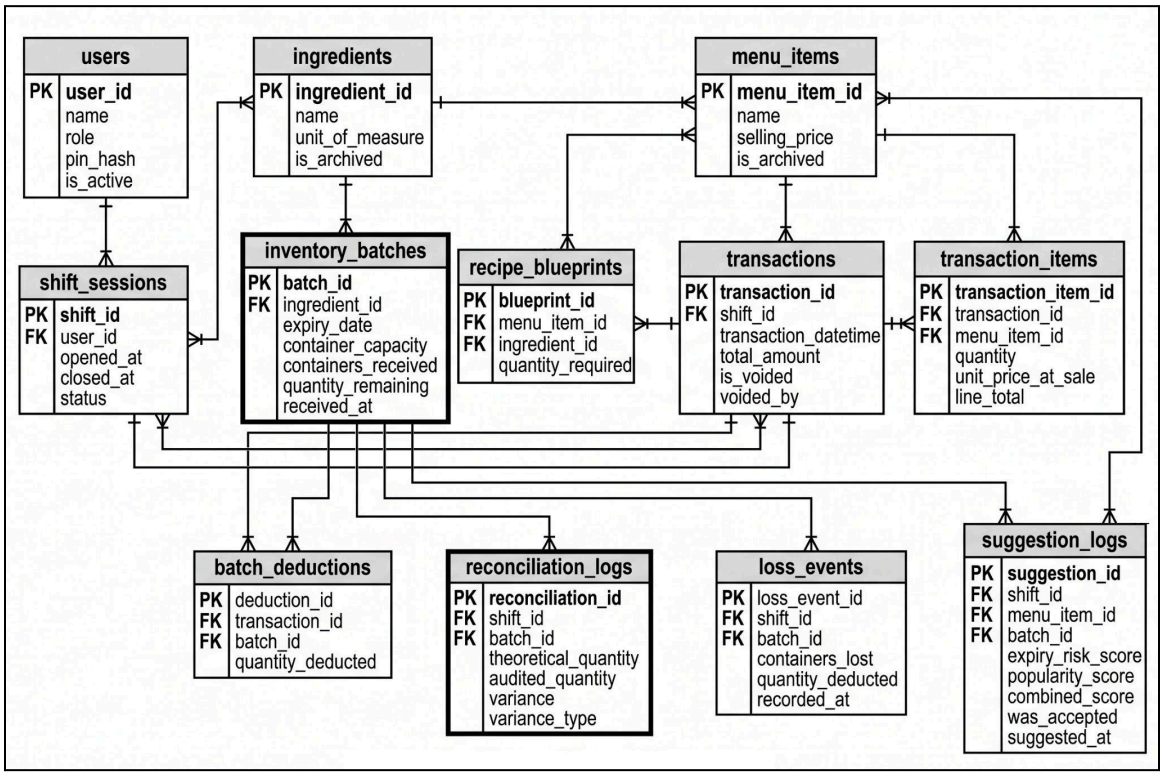


Figure 3.8 Entity Relationship Diagram

Data Dictionary

The data dictionary lists every field in the BrewManager database, along with its data type, constraints, and purpose. Because the system runs on SQLite, the types used are the ones SQLite supports directly: INTEGER for whole numbers and identifiers, REAL for quantities that may include fractions,

TEXT for names and timestamps, and INTEGER used as a boolean where a field records a yes or no state.

Three conventions apply throughout. Quantities are stored in the base unit of measurement recorded on the ingredient, so that all arithmetic within a single ingredient is consistent. Timestamps are stored as TEXT in ISO 8601 format, which SQLite can sort and compare directly. Records that must be preserved for historical accuracy are marked with an archive flag rather than being deleted, so that past transactions and deductions continue to reference valid rows.

Field Name	Data Type	Constraint	Description
user_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each user account
name	TEXT	Not Null	Display name of the staff member
role	TEXT	Not Null	Access level of the account, either Manager or Barista
pin_hash	TEXT	Not Null	Hashed value of the user's PIN so the PIN is never held in plain text
is_active	INTEGER	Not Null, Default 1	Indicates whether the account is currently in use

Table 3.2 Data Dictionary Table (users)

Field Name	Data Type	Constraint	Description
shift_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each shift session
user_id	INTEGER	Foreign Key, Not Null	References the user who opened the shift
opened_at	TEXT	Not Null	Timestamp when the shift began
closed_at	TEXT	Nullable	Timestamp when the shift was closed through reconciliation
status	TEXT	Not Null	Current state of the shift, either open or closed

Table 3.3 Data Dictionary Table (shift_sessions)

Field Name	Data Type	Constraint	Description
ingredient_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each raw ingredient
name	TEXT	Not Null, Unique	Name of the ingredient
unit_of_measure	TEXT	Not Null	Base unit used for all quantities of this ingredient, such as milliliters or grams
is_archived	INTEGER	Not Null, Default 0	Marks the ingredient as retired while preserving its historical records

Table 3.4 Data Dictionary Table (ingredients)

Field Name	Data Type	Constraint	Description
batch_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each inventory batch
ingredient_id	INTEGER	Foreign Key, Not Null	References the ingredient this batch holds
expiry_date	TEXT	Not Null	Expiration date shared by all containers in this batch
container_capacity	REAL	Not Null	Full quantity held by one unopened container in this batch
containers_received	INTEGER	Not Null	Number of containers recorded when the batch was received
quantity_remaining	REAL	Not Null	Current quantity remaining in the batch, updated by deductions and by reconciliation
received_at	TEXT	Not Null	Timestamp when the batch was registered

Table 3.5 Data Dictionary Table (inventory_batches)

Field Name	Data Type	Constraint	Description
menu_item_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each menu item
name	TEXT	Not Null, Unique	Name of the beverage or item offered for sale
selling_price	REAL	Not Null	Current selling price

			of the item
is_archived	INTEGER	Not Null, Default 0	Marks the item as removed from the menu while preserving its sales history

Table 3.6 Data Dictionary Table (menu_items)

Field Name	Data Type	Constraint	Description
blueprint_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each blueprint entry
menu_item_id	INTEGER	Foreign Key, Not Null	References the menu item this entry belongs to
ingredient_id	INTEGER	Foreign Key, Not Null	References the required ingredient
quantity_required	REAL	Not Null	Amount of the ingredient needed to prepare one unit of the menu item

Table 3.7 Data Dictionary Table (recipe_blueprints)

Field Name	Data Type	Constraint	Description
transaction_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each completed sale
shift_id	INTEGER	Foreign Key, Not Null	References the shift session during which the sale occurred
transaction_datetime	TEXT	Not Null	Timestamp when the sale was completed
total_amount	REAL	Not Null	Total value of the sale

is_voided	INTEGER	Not Null, Default 0	Indicates whether the sale has been reversed
voided_by	INTEGER	Nullable	References the Manager account that authorized the reversal

Table 3.8 Data Dictionary Table (transactions)

Field Name	Data Type	Constraint	Description
transaction_item_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each line of a sale
transaction_id	INTEGER	Foreign Key, Not Null	References the sale this line belongs to
menu_item_id	INTEGER	Foreign Key, Not Null	References the menu item sold
quantity	INTEGER	Not Null	Number of units of the item sold in this line
unit_price_at_sale	REAL	Not Null	Selling price of the item at the moment of sale, stored so later price changes do not alter past records
line_total	REAL	Not Null	Total value of this line, calculated from quantity and unit price at sale

Table 3.9 Data Dictionary Table (transaction_items)

Field Name	Data Type	Constraint	Description
deduction_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each individual deduction

transaction_id	INTEGER	Foreign Key, Not Null	References the sale that triggered the deduction
batch_id	INTEGER	Foreign Key, Not Null	References the specific batch the quantity was taken from
quantity_deducted	REAL	Not Null	Amount deducted from this batch for this sale

Table 3.10 Data Dictionary Table (batch_deductions)

Field Name	Data Type	Constraint	Description
reconciliation_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each audited batch result
shift_id	INTEGER	Foreign Key, Not Null	References the shift session in which the audit was performed
batch_id	INTEGER	Foreign Key, Not Null	References the batch that was audited
theoretical_quantity	REAL	Not Null	Quantity the system calculated should remain, based on recorded deductions
audited_quantity	REAL	Not Null	Quantity derived from the physical count submitted by staff
variance	REAL	Not Null	Difference between theoretical and audited quantity, negative for loss and positive for surplus
variance_type	TEXT	Not Null	Category of the

			variance, recorded as unexplained, declared loss, or unrecorded receiving

Table 3.11 Data Dictionary Table (reconciliation_logs)

Field Name	Data Type	Constraint	Description
loss_event_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each declared mid-shift loss
shift_id	INTEGER	Foreign Key, Not Null	References the shift session in which the loss occurred
batch_id	INTEGER	Foreign Key, Not Null	References the batch the loss was taken from
containers_lost	REAL	Not Null	Number of containers lost, recorded in quarter-container steps
quantity_deducted	REAL	Not Null	Amount deducted from the batch, calculated from containers lost and container capacity
recorded_at	TEXT	Not Null	Timestamp when the loss was declared

Table 3.12 Data Dictionary Table (loss_events)

Field Name	Data Type	Constraint	Description
suggestion_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each suggestion issued
shift_id	INTEGER	Foreign Key, Not Null	References the shift session during which

			the suggestion appeared
menu_item_id	INTEGER	Foreign Key, Not Null	References the menu item that was suggested
batch_id	INTEGER	Foreign Key, Not Null	References the batch whose expiry risk triggered the suggestion
expiry_risk_score	REAL	Not Null	Normalized expiry risk value used in the scoring calculation
popularity_score	REAL	Not Null	Normalized trailing seven-day sales popularity used in the scoring calculation
combined_score	REAL	Not Null	Weighted result of the two scores, ranking the item against other available items
was_accepted	INTEGER	Not Null, Default 0	Indicates whether the suggested item was sold after the suggestion appeared
suggested_at	TEXT	Not Null	Timestamp when the suggestion was displayed

Table 3.13 Data Dictionary Table (suggestion_logs)

Context Diagram

The context diagram presents BrewManager as a single process and shows what enters and leaves the system across its boundary. Because the system runs entirely on one offline device, there are no external systems,

servers, or third-party services to represent. The only entities that interact with the system are the two user roles defined by the permission matrix.

The Manager supplies the configuration and stock data the system depends on. This includes ingredient profiles, menu items, recipe blueprints, and inventory batch records created during receiving. The Manager also supplies filter parameters for the analytics dashboard and authorization when a transaction void is requested. In return, the system provides the Manager with expiry status information, material loss reports, exported files, and database backups.

The Barista supplies the operational data generated during a shift. This includes customer orders, declared container losses, and the physical count submitted at the close of each shift. In return, the system provides the Barista with digital receipts, current menu availability, waste-prevention suggestions, and the variance results produced after an audit is submitted.

The device clock is shown as an external source rather than as a user, since expiry evaluation and FEFO routing depend on the current date but the system cannot influence or verify it. This is the only input the system receives that does not come from a person. Figure 3.9 presents the context diagram.

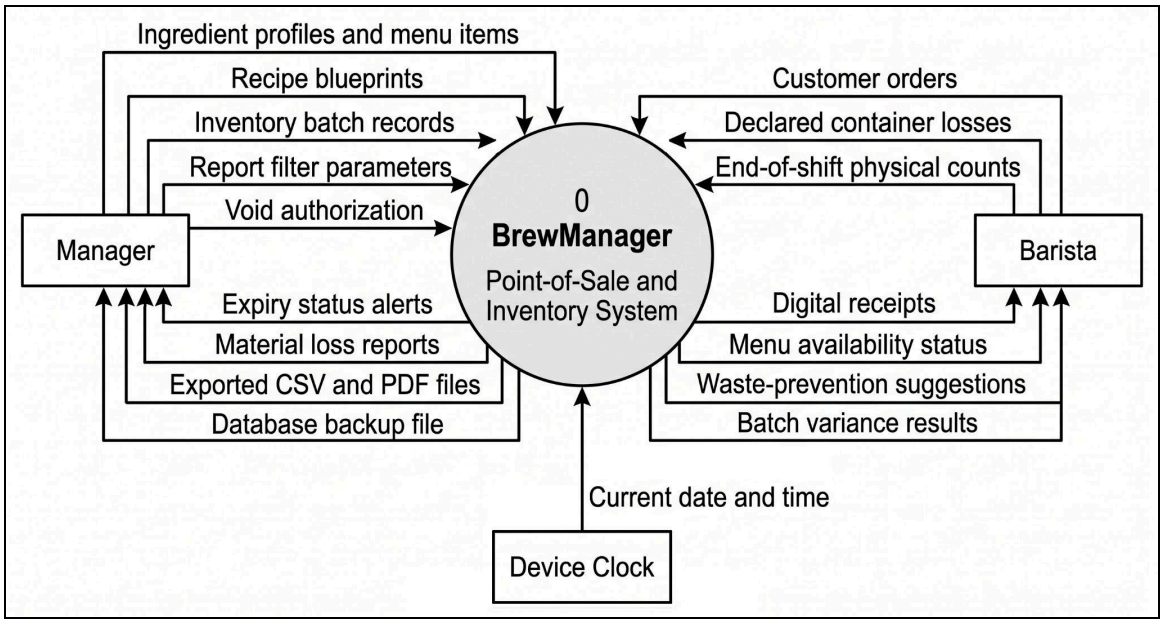


Figure 3.9 Context Diagram

Data Flow Diagram

The data flow diagram breaks the single process shown in the context diagram into the processes that make up the system, and shows the data stores each one reads from and writes to. Four processes are identified, corresponding to the main functions described under Requirements Modeling. The data stores shown group the database tables presented in the data dictionary according to how they are used.

Process 1, Setup and Configuration, receives configuration and stock data from the Manager. This covers PIN authentication records, ingredient profiles, menu items, recipe blueprints, and inventory batch registration. Configuration records are written to the configuration store, while batch registration writes one inventory batch record for each distinct combination of expiration date and container capacity in a purchase.

Process 2, Sales and Deduction, receives customer orders from the Barista. It reads recipe blueprints and prices from the configuration store, reads active batches from the inventory batches store, and applies FEFO routing with multi-batch splitting to determine which batches to deduct from. It updates the quantity remaining in the affected batches and writes the completed sale, its line items, and each individual deduction to the activity records store. This process also evaluates batch shelf-life against the device clock and produces the ranked waste-prevention suggestions returned to the Barista, along with digital receipts and current menu availability.

Process 3, Reconciliation and Correction, reads the inventory batches store to compute the theoretical quantity of every active batch, then receives the physical count submitted by the Barista. It converts the submitted container count and fractional value using each batch's container capacity, computes the variance against the theoretical quantity, and writes the corrected quantity back to the inventory batches store. Each result is written to the activity records store together with its variance type. Mid-shift losses declared through the Declare Container Lost function are handled by this process and deducted from the affected batch.

Process 4, Reporting and Backup, reads the activity records store to produce the material loss chart and the exported CSV and PDF reports for the Manager. It also produces a copy of the complete database file for backup.

The write-back performed by Process 3 is the flow that distinguishes this system from one that only records deductions. Because the corrected quantity is

written to the same inventory batches store that Process 2 reads from at the start of the next shift, each shift begins from a physically counted quantity rather than an estimate carried forward since the last correction. Figure 3.10 presents the data flow diagram.

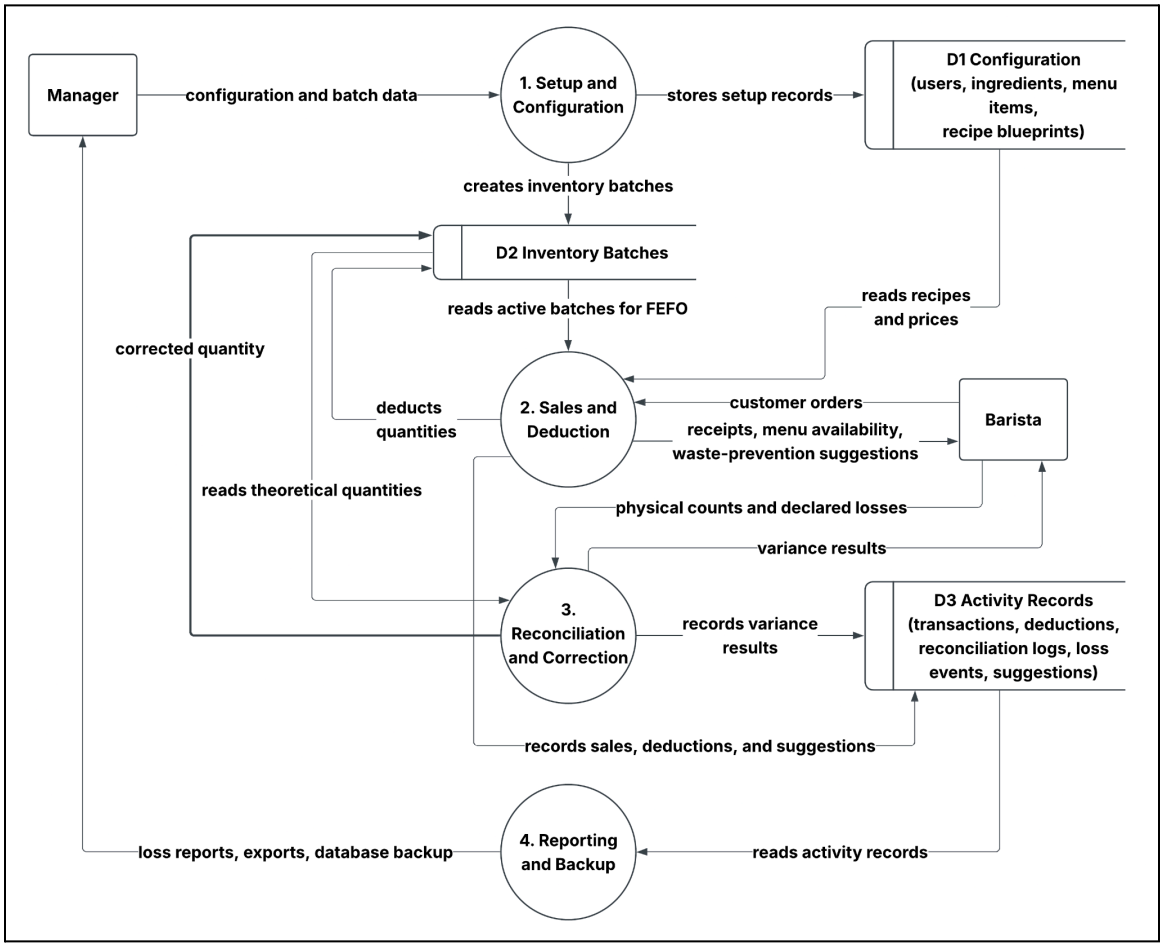


Figure 3.10 Data Flow Diagram

Object Modelling

This section describes the system in terms of its actors, its objects, and the order in which its operations occur. Three models are presented: the use case diagram, the class diagram, and the sequence diagram. A risk assessment follows.

Use Case Diagram

The use case diagram shows what each user role can do with the system. The two actors match the two access levels enforced by the permission matrix, so the diagram also serves as a visual record of what each role is and is not allowed to reach.

The Manager has access to all configuration and review functions. These include managing ingredient profiles, managing menu items, editing recipe blueprints, and registering incoming inventory batches. The Manager also views the expiry monitoring matrix, views the material loss dashboard, exports reports, backs up and restores the database, and manages Barista PINs.

The Barista has access to the operations performed during a shift. These include opening and closing a shift session, processing customer orders, declaring a container lost, and submitting the end-of-shift audit.

One use case is shared between the two actors. Voiding a transaction is initiated by the Barista at the POS screen but requires Manager authorization before it takes effect. It is therefore connected to both actors rather than placed under either one.

Several use cases include or extend others. Processing a customer order includes the automatic deduction of ingredients, since no order can be completed without it. Submitting the end-of-shift audit includes the calculation of variance and the write-back of corrected quantities to the affected batches. Voiding a transaction includes the restoration of deducted quantities to the batches they came from. These included use cases are drawn separately because they are

system operations rather than user choices, and because they are the operations that carry the system's inventory accuracy. Figure 3.11 presents the use case diagram.

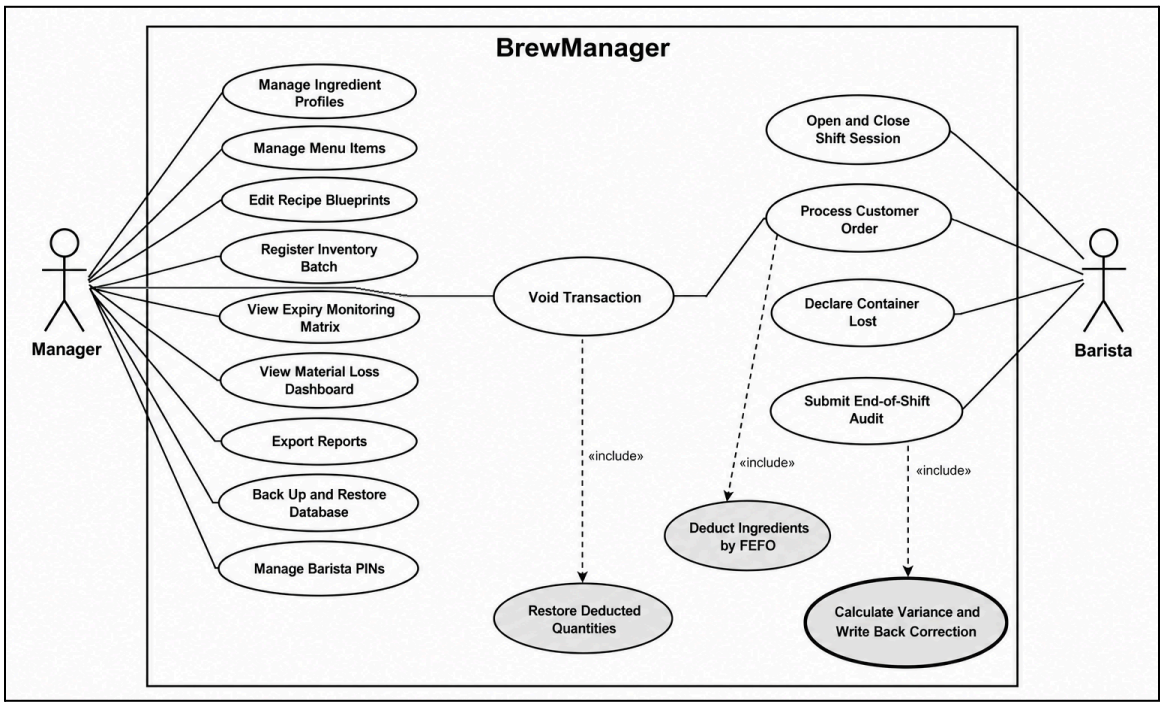


Figure 3.11 Use Case Diagram

Class Diagram

The class diagram shows the main classes that make up the application, the data each one holds, and the operations each one performs. The classes fall into three groups.

The first group is the model classes. These represent the records stored in the database and mirror the tables described in the data dictionary. Each model class holds the fields of its table and provides methods to convert between an object and a database row. Two model classes carry logic of their own. The inventory batch class evaluates its own expiry status against the current date and

returns a normalized expiry risk score, since these values depend only on the batch's own data.

The second group is the service classes. These hold the logic that operates on the models. The FEFO engine sorts batches by expiry date and performs deductions with multi-batch splitting. The transaction service creates sales records, records each deduction against the batch it came from, and reverses both when a void is authorized. The reconciliation service computes theoretical quantities, converts audit input using container capacity, calculates variance, and writes corrected quantities back to the affected batches. The decision support service calculates expiry risk and popularity scores and ranks menu items. Keeping this logic in service classes rather than inside the models allows each service to be tested on its own before it is connected to any screen.

The third group is the database helper. A single helper class manages the connection to the local SQLite database and provides the query methods the services depend on. It also provides the method that runs several writes as one database transaction, which is what allows a failed checkout to roll back every deduction rather than leaving ingredients deducted for a sale that was never recorded. Because the system runs on one device with no network layer, all data access passes through this one class.

Two design points are worth noting. The reconciliation service depends on the container capacity stored in the inventory batch, since the audit cannot convert a container count into a quantity without knowing how much one container holds. And the transaction service records each deduction against the

specific batch it came from, which is what allows a void to return each quantity to the correct batch.

Relationships between the model classes carry multiplicity labels showing how many instances of one class relate to another. The service classes and the database helper are shown without relationship lines, since they operate on objects passed to them rather than holding those objects as their own data. The order in which the services call one another is shown in the sequence diagram that follows. Figure 3.12 presents the class diagram.

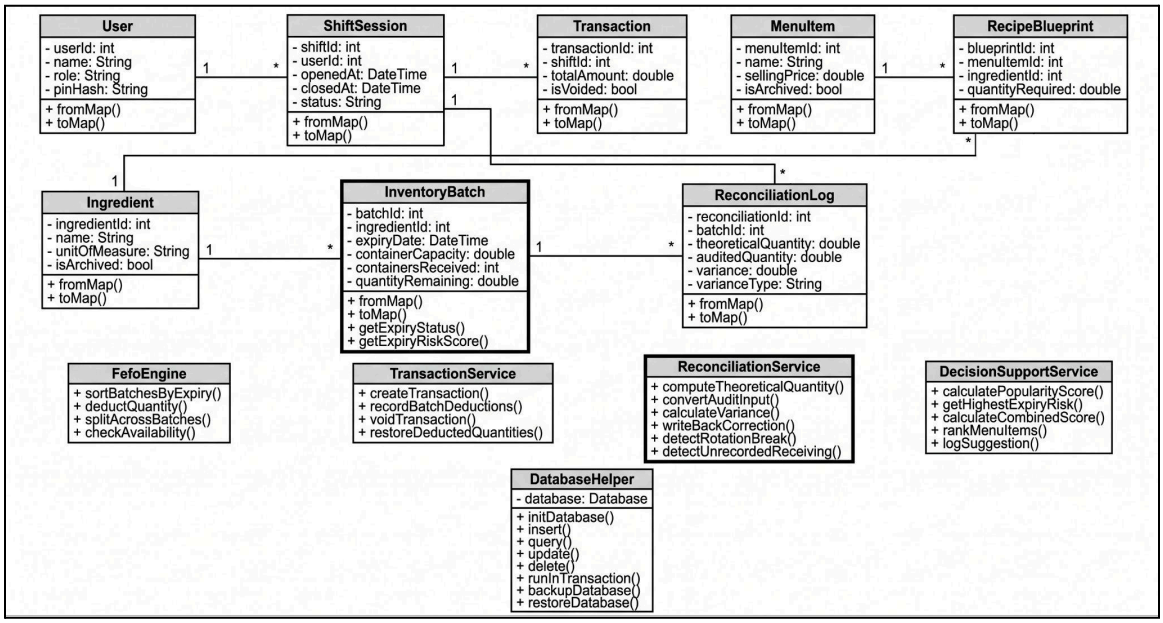


Figure 3.12 Class Diagram

Sequence Diagram

The sequence diagram shows the order in which objects call one another to complete an operation. Two sequences are presented, because the system has two operations that carry its main claims.

The first is the checkout sequence. When the Barista completes an order, the POS screen passes it to the transaction service, which opens a single

database transaction so that every write either completes together or is rolled back together. The service reads the recipe blueprint for each ordered item, then calls the FEFO engine to determine which batches to deduct from. The engine sorts the available batches by expiry date, deducts from the earliest-expiring batch first, and where that batch cannot cover the full amount, depletes it and carries the remainder to the next batch in order. Each deduction is written as its own record before the database transaction is committed. Once committed, the screen refreshes menu availability and requests updated suggestions from the decision support service.

The second is the reconciliation sequence. When the Barista closes a shift, the reconciliation service reads every active batch and computes its theoretical quantity, but does not display these values. The audit screen collects the sealed container count and the amount left in opened containers for each batch. Once the Barista submits, the service converts the submitted values using the container capacity stored on each batch, calculates the variance against the theoretical quantity, and writes the corrected quantity back to the batch record. It then writes the reconciliation log, checks for offsetting variances across batches of the same ingredient, and finally returns the results to the screen for display.

The order of steps in the second sequence is what enforces the blind audit. The theoretical quantity is computed near the beginning of the sequence but is not returned to the screen until after the count has been submitted and locked, so the Barista cannot adjust the count to match the system's expectation.

Figure 3.13 presents the checkout sequence and Figure 3.14 presents the reconciliation sequence.

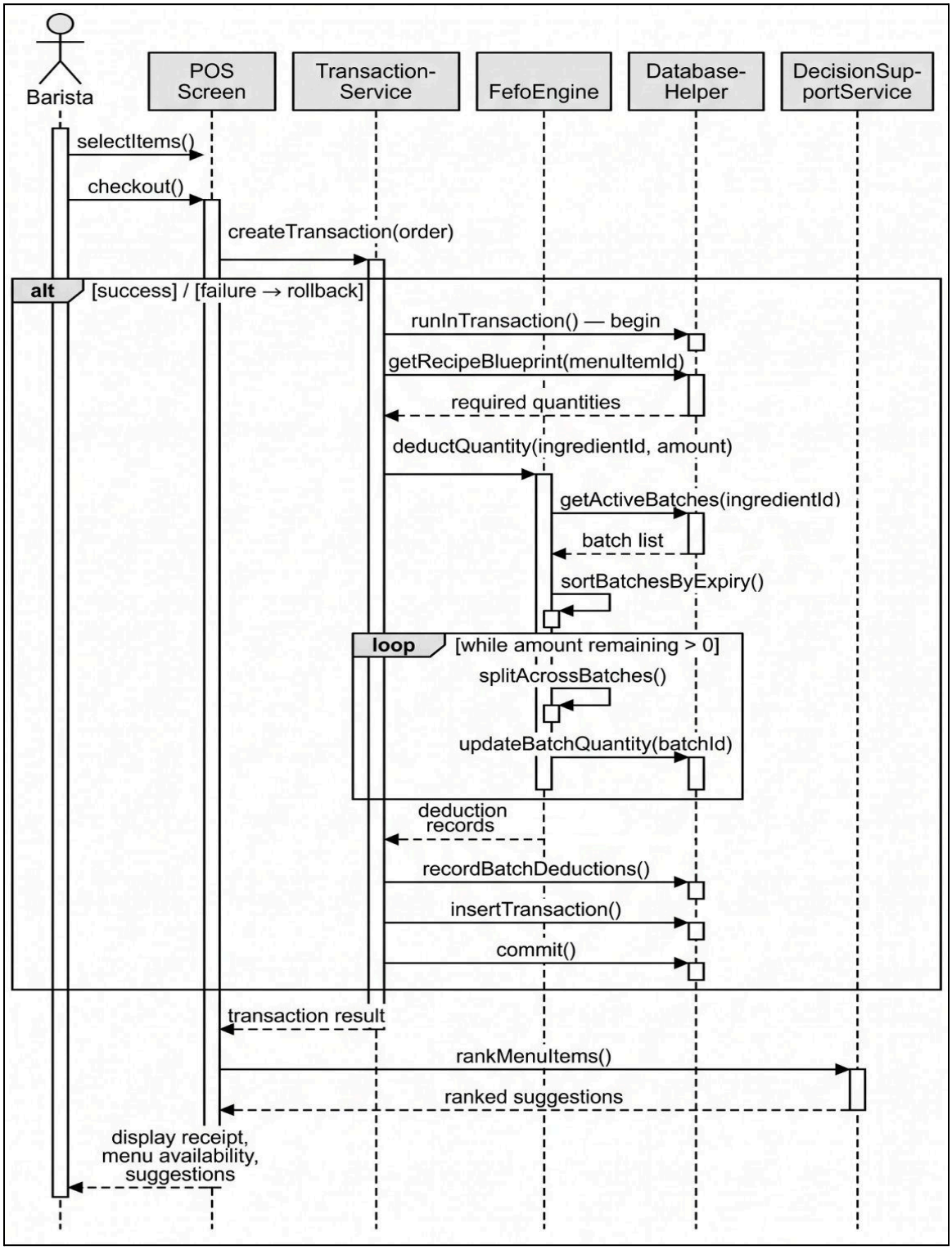


Figure 3.13 Sequence Diagram (Checkout)

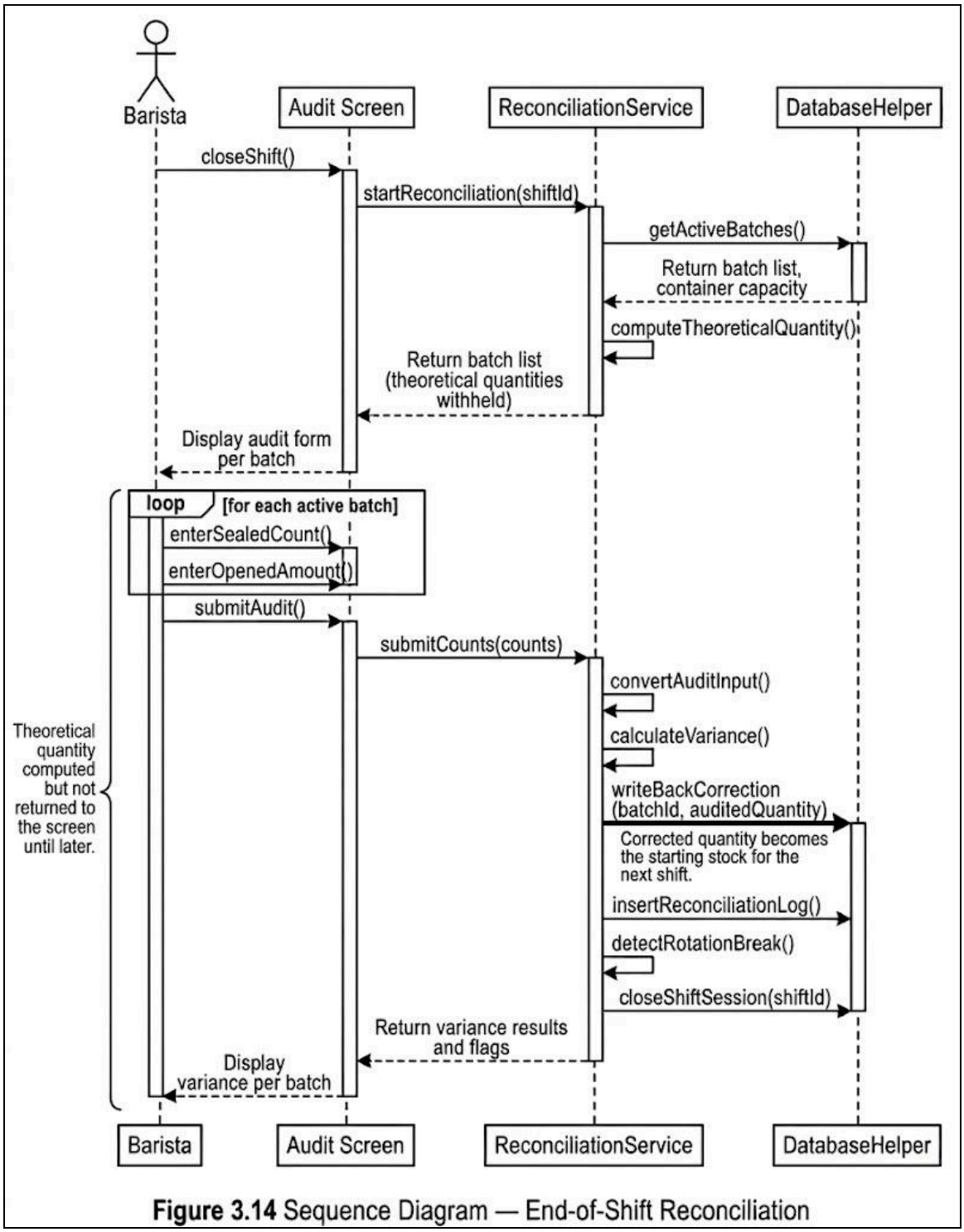


Figure 3.14 Sequence Diagram — End-of-Shift Reconciliation

Figure 3.14 Sequence Diagram (Reconciliation)

Risk Assessment / Analysis

This section identifies the risks that could affect the accuracy or availability of the BrewManager system, and describes how each one is addressed. The risks are grouped by what causes them: the offline single-device architecture, the dependence on staff input, and the boundaries of what the system can detect.

Risk 1: Total Data Loss from Device Failure

Because the system stores all records in a local SQLite database on one tablet, physical damage, theft, or a factory reset would destroy every transaction, inventory batch, and reconciliation record on the device. There is no cloud copy to fall back on.

This risk is addressed through the database backup function, which copies the complete SQLite file to external or user-selected storage. Unlike the CSV and PDF exports, which are reports and cannot be reloaded, this copy preserves the full state of the system and can be restored onto a replacement device. The residual risk is that backup is started by the user rather than run automatically, so any activity recorded after the last backup would be lost. This is reduced by prompting the Manager to back up at the end of each operating week.

Risk 2: Incorrect Device Clock

The system has no network access, so it depends entirely on the tablet's internal clock to evaluate expiry dates and to route deductions under FEFO. If the clock is set incorrectly, batches may be marked with the wrong shelf-life status and deductions may be routed to the wrong batch.

This risk is addressed by storing the latest timestamp the system has recorded and comparing it against the current device time at launch. If the clock appears to have moved backwards, the system displays a warning to the user. This detects the most common form of clock error, but the system cannot independently verify the correct time, so a clock set forward or set to a plausible but wrong date will not be caught. Keeping the device clock correct remains the responsibility of the cafe's management.

Risk 3: Human Error During Audit Input

The accuracy of the reconciliation module depends on staff entering the physical count correctly. A miscounted sealed container or a misjudged fractional estimate produces a variance that does not reflect what actually happened, and the incorrect value is then written back to the batch record.

This risk is reduced in three ways. The audit asks for only two inputs per batch and requires no weighing, unit conversion, or arithmetic, which limits the opportunities for error. Each control starts at the value recorded during the previous shift, so a routine closing requires only a small adjustment rather than entry from zero. And where a submitted sealed count exceeds the total containers recorded for that batch, the system requests confirmation before accepting the value, since such a count is physically impossible rather than merely unexpected. The residual risk is that a plausible but incorrect count will be accepted, since the system has no way to verify a physical count independently.

Risk 4: Incorrect Recipe Blueprints

All automated deduction depends on the recipe blueprints configured by the Manager. If a blueprint records the wrong quantity for an ingredient, every sale of that item deducts the wrong amount, and the resulting error accumulates until it is discovered.

This risk is treated as a configuration and measurement task rather than a software problem. Because independent coffee shops often prepare drinks by practiced estimation rather than measured amounts, time is set aside during development for on-site observation and measurement of actual pours. The reconciliation module also acts as a detection mechanism, since a consistently wrong blueprint produces a variance in the same direction on the same ingredient shift after shift, which appears as a pattern on the material loss dashboard.

Risk 5: Incorrect Receiving Entry

Expiry monitoring, FEFO routing, and reconciliation all draw on the batch records created by the Manager at receiving. If an expiration date is misread, a container capacity is entered incorrectly, or a purchase is not recorded at all, that batch will be tracked inaccurately for as long as it exists.

The system flags a counted quantity that exceeds recorded holdings as probable unrecorded receiving, which catches purchases that were never entered. It cannot, however, detect a mistyped expiry date or an incorrect container capacity, since both produce internally consistent records. This risk is reduced by keeping the receiving screen simple and by requiring the Manager to

enter each distinct expiry date and container size separately rather than in a single combined entry.

Risk 6: Deviation from Expiry-Order Stock Rotation

The system routes deductions to the earliest-expiring batch, but it cannot control which physical container staff take from the shelf. If staff open a later-expiring container while an earlier one remains, the system's records will no longer match which batch was actually consumed.

This risk is addressed through detection rather than prevention. Because reconciliation is performed per batch, using stock out of order produces a distinctive pattern: the batch the system assumed was consumed shows a surplus, while the batch actually used shows a deficit, and the two offset. The system flags this pattern for the Manager. Physical support is also provided by labeling containers with their batch at receiving and arranging stock so that earlier-expiring batches are positioned for first access. The residual risk is that a rotation break occurring alongside material loss of similar size may not be distinguishable from ordinary waste.

Risk 7: Unauthorized Access to Configuration

Because the system runs on a shared device behind the counter, an unauthorized user could reach configuration screens, alter recipe blueprints, or reverse transactions if access is not controlled.

This risk is addressed through the PIN-based permission matrix. Baristas can reach only the POS screen, the loss declaration, and the audit interface. Configuration, receiving, voiding, analytics, and data preservation are reserved

for the Manager account. PINs are stored as hashes rather than plain text, the default Manager PIN seeded at installation must be changed before any other function becomes available, and an idle session lock returns the application to the PIN entry screen after a period of inactivity.

System Design

This section describes the interface layout of the BrewManager system and the reports it produces. The design follows the two access levels enforced by the permission matrix, so the screens available to each role are described separately.

User-Interface

The application is built for an Android tablet in landscape orientation. This layout was chosen because the POS screen needs to display a menu grid and a running order at the same time, and because the audit screen needs to show several batch entries at once without scrolling through a narrow column.

The interface is organized around the two roles. After a user enters a PIN, the system routes a Manager account to the back-office dashboard and a Barista account to the POS transaction screen. Neither role can reach the other's screens.

The Manager screens cover configuration and review. These are the ingredient management screen, the menu item management screen, the recipe blueprint editor, the inventory receiving screen, the expiry monitoring matrix, the material loss dashboard, the export and backup screen, and the user

management screen. The receiving screen allows several batch entries to be recorded in one session, since a single purchase may contain containers with different expiry dates or container sizes.

The Barista screens cover shift operations. These are the POS transaction screen, the declare container lost dialog, and the end-of-shift reconciliation screen. The POS screen shows the menu as a grid of items with the running order beside it. Items whose ingredients have reached zero stock are grayed out and cannot be selected. Waste-prevention suggestions appear on the same screen together with the expiry condition that triggered them, so the barista sees why an item is being suggested rather than only which one.

The reconciliation screen presents one block per active batch. Each block shows the ingredient name, the batch expiry date, and the container capacity, so staff can identify which physical containers belong to which batch. Each block accepts two inputs: a count of sealed containers and a stepper for the amount left across all opened containers, in quarter-container steps. Theoretical quantities are not shown on this screen until the audit has been submitted.

Three design decisions apply across all screens. Controls are sized for touch input rather than for a mouse, since staff may be working quickly or with wet hands. Actions that change inventory records, such as voiding a transaction or submitting an audit, require a confirmation step. And color is used only to reinforce information that is also carried by text or position, so that expiry status remains readable without relying on color alone. The wireframes for each screen

are presented in Figures 3.15 through 3.2X.

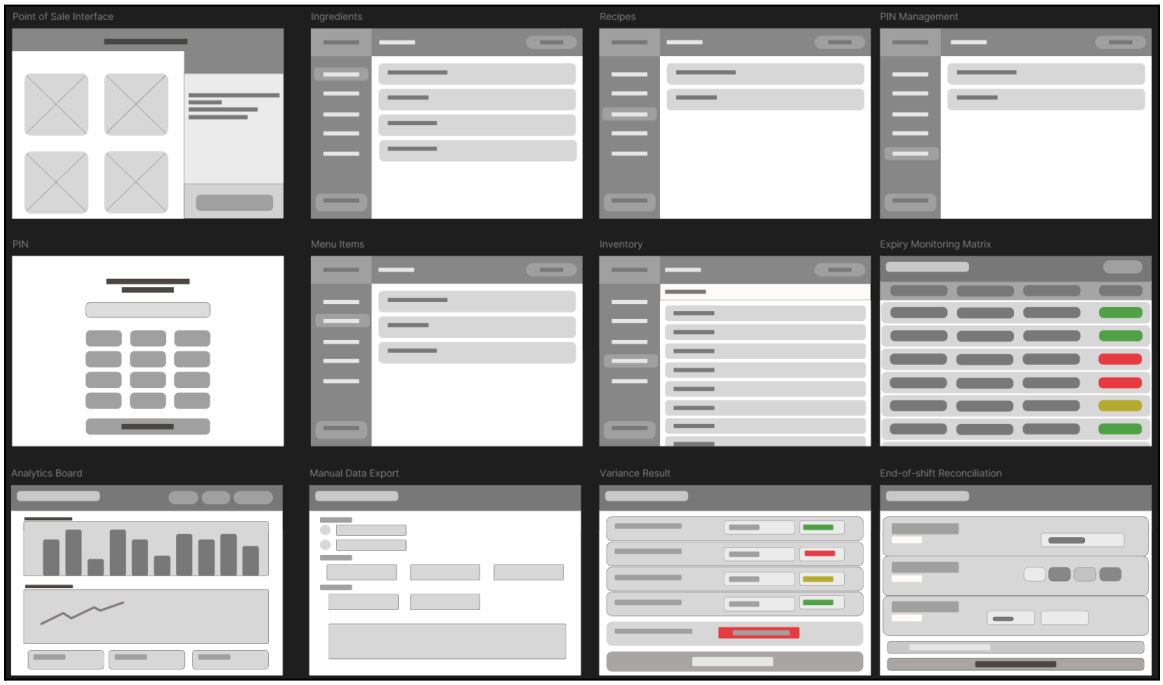


Figure 3.14 BrewManager Wireframes

Reports

The system produces two kinds of output. On-screen views allow the Manager to inspect current data within the application, and exported files allow records to be kept or printed outside it.

User Inquiry / View

The expiry monitoring matrix lists every active inventory batch with its ingredient name, expiry date, container capacity, and quantity remaining. Each row carries a color-coded status showing whether the batch expires in more than seven days, within seven days, or within forty-eight hours or sooner. The list is sorted by expiry date so the most urgent batches appear first.

The material loss dashboard presents variance per ingredient over time, drawn from the reconciliation logs. The Manager can filter by date range and by ingredient. Loss is shown by category so that unexplained variance can be told apart from declared losses such as dropped containers. Surpluses arising from unrecorded receiving are excluded from loss totals, since these are record corrections rather than material waste.

After each reconciliation is submitted, the system displays the result per batch, showing the theoretical quantity, the counted quantity, and the variance between them, together with any rotation or unrecorded receiving flags raised during the audit.

Print-out Report

The Manager can export three reports in CSV or PDF format to the tablet's internal storage.

The sales report lists completed transactions over a selected date range, showing the date and time of each sale, the items sold, the quantity of each, the price at the time of sale, and the transaction total. Voided transactions are included and marked as such rather than removed, so that the record remains complete.

The inventory report lists all active batches with their ingredient, expiry date, container capacity, containers received, and quantity remaining, allowing current stock to be reviewed away from the device.

The reconciliation report lists audit results over a selected date range, showing the shift, the batch audited, the theoretical and counted quantities, the variance, and the variance type. This is the report a Manager would use to review material loss over a longer period than the dashboard displays.

The system does not print physical receipts. Order summaries are shown on screen at the end of each transaction, which removes the cost of a receipt printer and of consumable paper. Sample layouts for each report are presented in Figures 3.2X through 3.2X.

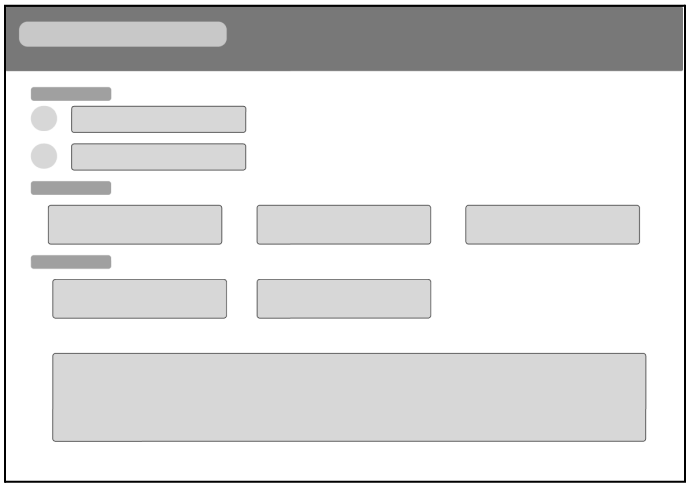


Figure 3.18 Manual Data Export Interface

System Development

Technical Feasibility

Technical feasibility examines whether the BrewManager system can be built and run with the tools, skills, and hardware available to the proponents and the client. The assessment covers the minimum hardware and software needed to run the system, whether those components work together, and why each technology was chosen.

The system is technically feasible because it uses established, freely available tools that the proponents are already familiar with, and because it runs on a single mid-range Android tablet rather than on a server or a network. No component of the system requires specialized hardware, paid licensing, or an internet connection.

Minimum Requirements

The following table presents the minimum hardware and software requirements for running BrewManager on the target deployment device.

Category	Requirement	Minimum Specification
Client Hardware	Device type	Android tablet
	Android version	Android 8.0 (Oreo) or higher
	Processor	Quad-core 1.4 GHz or equivalent
	Memory (RAM)	2 GB or higher
	Internal storage	8 GB available
	Display Size	8 inches or larger
	Display Resolution	1280 × 800 pixels
	Network	Not required
Development Hardware	Device Type	Laptop or desktop computer
	Operating System	Windows 10 (64-bit) or later, macOS 11 or later
	Processor	Intel Core i5 or equivalent
	Memory (RAM)	8 GB
	Internal Storage	20 GB available
	Testing Device	Physical Android tablet meeting client hardware specification
Development Software	Framework	Flutter SDK 3.0 or later

	Programming Language	Dart 3.0 or later
	Integrated Development Environment	Android Studio or Visual Studio Code
	Android Build Tools	Android SDK, API Level 26 or higher
	Database Package	sqflite
	Charting Package	fl_chart
	File Handling Packages	path_provider, csv, pdf
	Version Control	Git

Table 3.14 Minimum Requirements Table

Compatibility Checking

Compatibility was checked in three areas: between the software components, between the software and the target device, and between the system and the client's existing way of working.

The software components are compatible because they belong to the same ecosystem. Flutter compiles to native Android code, Dart is the language Flutter uses, and the SQLite packages selected are maintained for Flutter and are widely used in production applications. No component depends on a service that must be reached over a network, so nothing in the stack fails when the device is offline.

The choice of Android 8.0 as the minimum version was made after checking the version requirements of the packages used. Setting the minimum at this level covers the large majority of Android tablets currently in use while remaining within the supported range of every package in the project. Testing is carried out on a physical tablet matching the minimum specification rather than

only on higher-end hardware, so that performance on the weakest supported device is known rather than assumed.

Compatibility with the client's existing operations was also considered. The system does not require the cafe to change suppliers, container sizes, or preparation methods. It does not require a digital scale, a receipt printer, or a network connection. The only new practice it introduces is labeling containers with their batch at receiving, which supports a stock-rotation habit that food service already follows.

Relevance of the Technologies

Each technology was chosen for a specific reason rather than for familiarity alone.

Flutter was chosen because it produces a native Android application from a single codebase and provides a widget set suited to touch interfaces. Its layout system handles the landscape tablet screens the system depends on, and its state management allows the menu grid to update the moment a batch reaches zero stock, which is what makes the reactive grayout possible.

Dart is the language Flutter uses. Its asynchronous handling is relevant to this system because checkout involves several database writes that must complete in order, and its support for structured error handling allows a failed checkout to be caught and rolled back rather than leaving ingredients deducted for a sale that was never recorded.

SQLite was chosen because it is a relational database that runs entirely on the device with no server. This matters for two reasons. The system's data

model depends on relationships between batches, deductions, transactions, and reconciliation logs, which a simple key-value store could not represent. And SQLite supports grouping several writes into one transaction, which is what allows a multi-item checkout with multi-batch splitting to either complete fully or roll back fully.

fl_chart was chosen for the material loss dashboard because it renders charts locally without contacting an external service, which keeps the system consistent with its offline design.

Version control through Git was used throughout development. Because the system is built in five dependent increments, being able to return to a known working state after a failed change was necessary rather than optional.

Taken together, these choices support the system's central requirement: that a small independent coffee shop can run a complete point-of-sale and inventory system on one affordable device, without a server, a subscription, or a network connection.